

A Consistent Parallel Estimation Framework for Visual-Inertial SLAM

Zheng Huai , *Member, IEEE*, and Guoquan Huang , *Senior Member, IEEE*

Abstract—In this article, we revisit the optimal fusion of visual and inertial information from a monocular camera and an inertial measurement unit and propose a novel parallel visual-inertial simultaneous localization and mapping (SLAM) estimation framework in favor of the multithread computation on a single CPU. We start modeling the SLAM problem with a Bayesian batch estimator, and then split it into two submodules, localization and mapping, of different scales and processing rates, however, can thus run concurrently. The estimation consistency is taken into account in decoupling the two submodules so that when loop closure occurs the localization accuracy can seamlessly benefit from the mapping result via online global optimization, which distinguishes our solution from the others. To this end, we design the corresponding front-end and back-end to consistently solve localization and mapping in parallel, especially the hybrid robocentric and world-centric formulations are used for modeling the respective problems. We also demonstrate the effectiveness of the proposed method using both the synthetic data generated for Monte-Carlo simulations and diverse real datasets acquired in highly-dynamic, long-term, and large-scale SLAM scenarios. Simulation results validate the significantly improved consistency and accuracy by applying our method. Experimental results show the better (competitive at least) performance against a state-of-the-art method, while being capable of processing a huge amount of measurements in building large-scale maps *without* blocking the high-accuracy real-time localization outputs.

Index Terms—Estimation consistency, hybrid formulation, parallel computing, visual-inertial (VI) simultaneous localization and mapping (SLAM).

I. INTRODUCTION

SIMULTANEOUS localization and mapping (SLAM) has been studied over three decades since the pioneer works (e.g., [1], [2], and [3]) were proposed around the 1990s, and has been the dominant technology for many different computer vision and robotics applications, such as the recently popular augmented/virtual/extended reality (AR/VR/XR) and autonomous driving. Generally speaking, SLAM is used to let the sensing

platform (e.g., a robot) learn about its own states, such as the immediate pose and velocity with respect to a reference frame, and its surrounding environment which is usually represented by a map. However, as those states in general are not directly observable we have to infer them based on the measurements that come from the onboard sensors, where the wheel odometer [4], sonar [5], light detection and ranging (LiDAR) [6], [7], camera [8], [9], and inertial measurement unit (IMU) [10], [11] are most commonly used in the real applications. Some of the SLAM methods depend on a single type of sensor, such as the LiDAR and camera, while some others prefer to fuse multiple sources of information by using different sensor combinations, among which the LiDAR-IMU (e.g., [12] and [13]) and camera-IMU (e.g., [14], [15], and [16]) are specially adopted in favor of leveraging the rich 3-D/2-D environmental information from the LiDAR scans and camera images with the fast motion sensing ability of IMU to improve the accuracy of the output map and trajectory. Although apart from the autonomous vehicles some compact and lightweight LiDAR sensors have started showing on the consumer-grade devices, like the smart phones, camera is still the most prevalent choice for visual sensing and keeps evolving with time. Regarding this fact, in this article, we focus on the SLAM solution using camera and IMU as the sensors. In particular, the *minimal sensing*, that includes a monocular camera and a single IMU, is enforced in our solution in order to possess better versatility and the extensibility as well.

Practical sensors notably suffer from the time-variant electronic/magnetic noises and biases whose emergence may depend on the materials used or manufacturing of the sensors. In order to extract usable information from their noisy outputs to infer the state of the sensing platform, usually the probabilistic models (e.g., assuming the noise is Gaussian) are used to characterize the corresponding measurements. Based on these models and some prior knowledge (in terms of the probability distribution) of the state, the posterior probability of the state can be obtained following the Bayes rule [17]. In particular, the maximum of this probability corresponds to the (optimal) inference of the state, thus named the maximum-a-posteriori (MAP) estimation. It is well known that under certain assumptions finding the MAP estimate of the state of SLAM can be cast as a nonlinear least-squares (NLS) problem which is also referred to as bundle adjustment (BA) in the literature, with the solution usually in batch form (e.g., [18] and [19]). However, the computational cost of batch estimation is growing as more sensor measurements are included, thus may eventually become prohibitive for real-time

Manuscript received 20 January 2024; revised 16 May 2024; accepted 26 June 2024. Date of publication 25 July 2024; date of current version 9 August 2024. This work was supported in part by the University of Delaware (UD) College of Engineering and the NSF under Grant IIS-1924897. This article was recommended for publication by Associate Editor O. Stasse and Editor S. Behnke upon evaluation of the reviewers' comments. (*Corresponding author: Zheng Huai.*)

Zheng Huai is with the Department of Mechanical Engineering, University of Delaware, Newark, DE 19716 USA (e-mail: zhuai@udel.edu).

Guoquan Huang is with the Department of Mechanical Engineering, Department of Computer and Information Sciences, University of Delaware, Newark, DE 19716 USA (e-mail: ghuang@udel.edu).

Digital Object Identifier 10.1109/TRO.2024.3433868

processing, even for some incremental alternatives [20], [21]. In order to achieve efficiency against the ever-increasing computational complexity, the approaches of filtering or fixed-lag smoothing are proposed (e.g., [14] and [22]) which focus on the estimation over only a bounded-size, sliding window of recent states by marginalizing the past states and measurements periodically. While those methods achieve constant processing cost and real-time performance, they still suffer ever-increasing pose drift in the estimation due to the inability to perform global adjustment when *loop closure* (e.g., the features mapped at an early time are reobserved at the same place) occurs.

To overcome these limitations, parallel tracking and mapping (PTAM) [23] is proposed, which splits a SLAM problem into two subones: 1) motion tracking; and 2) map building, so as to solve them separately in parallel, and later becomes a popular architecture among SLAM systems. Its advantage is straightforward because the two pipelines are decoupled and run independently, thus leading to an efficient solution. Inspired by that, most SLAM systems nowadays are designed with 1) a fast *front-end* for estimating the immediate pose of the sensing platform and some locally observed features; and 2) a relatively slow *back-end* for jointly optimizing the past poses and features in the map (e.g., [9], [24], and [25]). In this way, the loop-closure measurements can be processed as the past states are retained in the back-end, and according to the aforementioned works this can be summarized as two stages: 1) relocalization is performed first by the front-end using the map built by the back-end; and 2) loop-closure constraints are then included into the back-end for a batch optimization of the map. While this achieves global adjustment functionally, the decoupling mechanism used *ignores* a fact that the states in the front-end and back-end are probabilistically correlated. As pointed by [21] and [26], the map queried in stage 1) has to be fixed during the fusion, which may easily bias or even crash the estimation if the linearization errors of those (past) features involved are significant. In practice, this shall directly degrade the consistency and accuracy of PTAM-based SLAM, thus demanding a theoretically rigorous parallel solution.

In this article, we perform sliding-window motion tracking in the front-end and batch optimization in the back-end, but different from the PTAM-based approaches, here: 1) the loop-closure measurements are not processed by the front-end *but* deferred to the back-end; and 2) the back-end optimizes (all) the states including both the past states that have been marginalized from the front-end *and* the current states that are being estimated by the front-end, by fully preserving their correlations. As a result, the front-end of our visual-inertial (VI)-SLAM only focuses on exploration by performing visual-inertial odometry (VIO) to provide immediate estimates of the most recent states, while the back-end performs batch SLAM with loop closures using asynchronous global optimization. As the state evolution in the front-end is tracked in real-time by the back-end, the obtained global adjustment can be applied to both ends simultaneously and consistently.

Thanks to this novel functional decoupling, we are free to use different VIO in the front-end, while seriously taking into account estimation consistency in the design of asynchronous

mechanism for the back-end global optimization. In particular, the VIO in our front-end uses *robocentric* formulation [16] for improved estimation consistency, while our back-end realizes a *world-centric* BA to facilitate global map-based use cases, such as 3-D reconstruction and place recognition. As such, the proposed parallel VI-SLAM system is able to run on a single CPU by leveraging its multicore processing ability, which is later validated on synthetic data generated for the Monte-Carlo simulations and real-world datasets of highly-dynamic, long-term, and large-scale SLAM scenarios.

II. RELATED WORK

In this section, we briefly discuss the recent representative SLAM methods for processing visual and IMU measurements. As a popular topic, there have been many existing approaches, and we recommend recent survey [27] for a big picture of all and another one [28] summarizing the efforts in visual-inertial navigation systems. Here, we prefer to consider VI-SLAM a superset of VIO mainly because of its ability of self-correction through loop closures. Also, we focus on the tightly coupled solutions which directly fuse visual and inertial measurements for state estimation, whose performances are typically superior over the loosely-coupled counterparts (e.g., [29] and [30]) which process visual and inertial measurements separately and then use these results for inferring the state estimates.

In the early decades after the SLAM problem was proposed, the filtering and BA methods were used for jointly estimating the poses and observed features of the sensing platform [31], [32]. However, as the technology advances the high frequency sensors, such as the camera and IMU, have started to be used for improving the SLAM accuracy. While those sensors are able to provide richer environmental information and more precise motion description, the corresponding huge amount of measurements directly increases the computation of a SLAM estimator. Especially, for an extended Kalman filtering (EKF)-based estimator, it has computational complexity quadratic to the number of features estimated. Even though the camera may only observe a small number of features at each timestep, the covariance update has to include all the features and poses due to their correlation. Instead, the BA-based estimator relies on information (i.e., Hessian) matrix whose sparse pattern can be used to speed up its update. However, the cubic computational complexity in solving the state estimates (e.g., using Cholesky decomposition [19]) will easily make SLAM intractable when more features and poses are included in the estimation.

One family of the solutions is to reduce the number of state estimated by marginalizing the past states periodically, among which the well known two should be the sliding-window EKF (e.g., [16], [33], [34], and [35]) and the fixed-lag smoothing (e.g., [11], [15], [22], and [36]). Some recent works only estimate the poses of the sensing platform by exploiting multiple-view constraints in consecutive feature observations (e.g., [11], [14], and [16]), while others still follow the SLAM-based formulation to estimate both poses and observed features (e.g., [15] and [34]). In particular, some efforts also combine the above two paradigms for better performance, such as [22] and [37].

Nevertheless, those approaches are usually referred to as VIO in the literature due to the function similar to an odometry. Also, because no past state information is kept those estimators cannot perform loop closure when reobserving the previously seen features.

On the other hand, PTAM [23] is proposed to overcome the limitations of the original batch EKF and BA-based methods. It splits SLAM into motion tracking and map building and run them in two separate threads so that the slow mapping process will not block real-time pose outputs. In particular, the map is used as a prior to refine motion tracking results, and the concept of keyframe is proposed so as to reduce the size of map by only keeping the critical poses and features. As this seminal work focuses on the small AR scenario where a large portion of features can be tracked all the time, loop closure is not investigated. However, its design has been inherited by many later works, especially on visual SLAM (e.g., [9], [24], and [38]). Loop closure detection is enabled in most of these works and global optimization methods are also proposed. Basically, following PTAM, loop-closure measurements are first used on localizing current camera frame, along with the other feature measurements. Then, once this frame is selected as keyframe and added in the map the loop-closure measurements will be used for a batch optimization including all the keyframes. In general, this strategy is shown functional in practice by those works and is adopted by their follow-ups (e.g., [39] and [40]) and other recent visual-inertial SLAM systems (e.g., [41] and [42]).

Almost in the meantime, research efforts also try to exploit online capability of batch filtering or BA-based solution. One of the threads uses square root information-based formulation and proposes an incremental update method based on the QR factorization [20], [43]. Especially, variable reordering is used to keep the sparseness of the square root information matrix so that the fusion of loop-closure measurements and solving for the state update become more efficient. Compared to [20] that performs periodically batch relinearization, [43] realized fluid relinearization by using the proposed Bayes tree structure [44]. While these approaches significantly improve the efficiency of batch estimator, they still suffer from increased computational complexity, especially with the dense constraints (e.g., number of measurements at least five times the number of poses [43]) that can be commonly seen in the visual(-inertial) applications. For filtering-based methods, Schmidt estimator is investigated in both covariance and information domains [21], [45]. In their strategies only the most recent states are being updated in real-time while leaving the other states constant or asynchronously updated. In particular, the correlation between the above two parts of the states is maintained and jointly updated so that loop-closure measurements are able to be processed. Those Schmidt estimators achieve linear computational complexity in the total number of states estimated, and most importantly they have improved estimation consistency [46], [47]. However, as a SLAM estimator, one notable limitation of those approaches is that relinearization is hard to perform or risks high cost for real-time performance, thus limiting their potentials in gaining higher accuracy, especially for the mapping. Also, as Schmidt

estimator is a conservative approximate of the batch estimator, it can only provide suboptimal state estimates. There are also efforts to integrate filtering and BA-based solutions [48], [49]. Both above works are based on [43], where a batch estimator was partitioned and solved by filtering and BA (smoothing) in parallel with respect to different parts of the states. In this way, the loop-closure measurements can be processed, however, in a delayed manner, as it needs the states involved (e.g., poses) move from the filtering part into the BA part so that closing the loop by BA is independent of the process of filtering. As the correlation of states between the filtering and BA parts is kept as in a batch estimator, once loop closure is done by BA the states in the filtering part get updated simultaneously.

To achieve the optimality of batch SLAM estimator with the information of loop closure meanwhile guarantee the real-time performance, [50], and our previous work, [26], are proposed very recently. Despite of different implementations of systems, similar parallel designs are advocated. In particular, the front-end (or client) focuses on the immediate estimates of the most recent states, while the back-end (or server) maintaining the entire states will handle loop closures. As the state correlation between front and back-ends is always kept and being updated in real-time, optimal global adjustments after fusing the loop-closure measurements can be applied to both ends, similar to a batch estimator, while an exception in [50] is when correcting the front-end state estimate an approximate prior that ignores the correlation between the states involved in the loop-closure measurements is used. The algorithm in [50] is presented in a general form and developed with a specific realization of [43]. Unfortunately, [50] does not show any implementation details. Our system [26] is based on a square-root information filtering method that, as we discussed earlier, may not be efficient for relinearization that is crucial for improving mapping accuracy. Also, it is an approximate solution due to losing information in the batch optimization. Therefore, in this article, we switch to the general graph-based SLAM to develop an exact solution, and present in detail about our implementation. Especially, we aim to propose a new SLAM framework which: 1) is friendly to the computation resources by decoupling the functionalities; 2) provides accurate state estimates by consistently leveraging the loop-closure information; and 3) can be implemented without limiting to any specific software or platform.

III. BATCH SLAM ESTIMATOR

In this section, we recap the SLAM problem in the form of graph (of nodes and edges) and its batch solution based on the maximum likelihood estimation (MLE) or MAP estimation (if some prior knowledge is available).

Typically, the states of a vision-based SLAM problem will include the poses of the sensing platform at each timestep and the positions of the features observed from the corresponding poses.¹ Each of them can be represented by a *node*, $\mathbf{x}_i$, with i the index. In particular, if we define $\mathcal{I}$ as the set of indexes for

¹When IMU is used, the states in the pose (orientation and position) can be augmented with the velocity and biases [14] so as to describe the motion of the sensing platform more precisely.

all the nodes in a graph, the states can be denoted as

$$\mathcal{X} = \{\mathbf{x}_i\}_{i \in \mathcal{I}}. \quad (1)$$

The inputs for a SLAM system are the measurements from the sensors, such as cameras and IMU. Like the *edges*, those measurements form the spatial constraints between the nodes: $\mathbf{z}_{ij} = \mathbf{h}(\mathbf{x}_i, \mathbf{x}_j) + \mathbf{n}_{ij}$, where $\mathbf{h}(\cdot)$ is the measurement model with $\mathbf{n}_{ij} \sim \mathcal{N}(\mathbf{0}, \Lambda_{ij})$ a zero-mean white Gaussian noise. As a whole, all of them can be denoted as

$$\mathcal{Z} = \{\mathbf{z}_{ij}\}_{(i,j) \in \mathcal{E}}. \quad (2)$$

With the definitions above, given all the measurements $\mathcal{Z}$ (edges), the likelihood function of the state $\mathcal{X}$ (nodes) is

$$p(\mathcal{Z}|\mathcal{X}) = \prod_{\mathbf{z}_{ij} \in \mathcal{Z}} p(\mathbf{z}_{ij}|\mathbf{x}_i, \mathbf{x}_j). \quad (3)$$

The maximum likelihood estimate of $\mathcal{X}$, that maximizes this probability density function, can mathematically equivalently be obtained by minimizing the negative logarithm of $p(\mathcal{Z}|\mathcal{X})$

$$\begin{aligned} \mathcal{X}^* &:= \arg \min_{\mathcal{X}} -\log p(\mathcal{Z}|\mathcal{X}) \\ &= \arg \min_{\mathcal{X}} \sum_{\mathbf{z}_{ij} \in \mathcal{Z}} \|\mathbf{z}_{ij} - \mathbf{h}(\mathbf{x}_i, \mathbf{x}_j)\|_{\Lambda_{ij}}^2 \end{aligned} \quad (4)$$

where we have used the notation: $\|\mathbf{e}\|_{\Lambda}^2 = \mathbf{e}^\top \Lambda^{-1} \mathbf{e}$ (i.e., the squared Mahalanobis norm of $\mathbf{e}$ with covariance Λ). It should be noted that $\mathbf{h}$ is usually a nonlinear function, and therefore (4) is a NLS problem. Note that, the iterative Gauss-Newton method is usually used for solving it. Specifically, we convert all the states into a single vector: $\mathcal{X} \mapsto \mathbf{x}$, and at k th iteration a correction, $\delta \mathbf{x}^{(k)}$, to the current estimate, $\hat{\mathbf{x}}^{(k)}$, is computed by minimizing the linearized version of (4)

$$c(\hat{\mathbf{x}}^{(k)} + \delta \mathbf{x}) \simeq \sum_{\mathbf{z}_{ij} \in \mathcal{Z}} \|\mathbf{H}_{ij}^{(k)} \delta \mathbf{x} - \mathbf{e}_{ij}^{(k)}\|_{\Lambda_{ij}}^2 = \|\mathbf{J}^{(k)} \delta \mathbf{x} - \mathbf{r}^{(k)}\|^2 \quad (5)$$

where each edge is linearized at the current state estimate, with its Jacobian matrix $\mathbf{H}_{ij}^{(k)} = \frac{\partial \mathbf{h}(\mathbf{x}_i, \mathbf{x}_j)}{\partial \mathbf{x}} \big|_{\mathbf{x}=\hat{\mathbf{x}}^{(k)}}$ and residual $\mathbf{e}_{ij}^{(k)} = \mathbf{z}_{ij} - \mathbf{h}(\hat{\mathbf{x}}_i^{(k)}, \hat{\mathbf{x}}_j^{(k)})$. Notably, with $\Lambda_{ij}^{-1/2}$ the matrix square root of Λ_{ij} , a *linear* least-squares problem is obtained by weighting and stacking all the Jacobians and residuals, as

$$\mathbf{J}^{(k)} = \begin{bmatrix} \Lambda_{01}^{-\frac{1}{2}} \mathbf{H}_{01}^{(k)} \\ \vdots \\ \Lambda_{ij}^{-\frac{1}{2}} \mathbf{H}_{ij}^{(k)} \\ \vdots \end{bmatrix} \quad \mathbf{r}^{(k)} = \begin{bmatrix} \Lambda_{01}^{-\frac{1}{2}} \mathbf{e}_{01}^{(k)} \\ \vdots \\ \Lambda_{ij}^{-\frac{1}{2}} \mathbf{e}_{ij}^{(k)} \\ \vdots \end{bmatrix}. \quad (6)$$

Since the sensor measurements typically outnumber the estimated states, $\mathbf{J}^{(k)}$ is a tall matrix, and QR factorization [51] can be used in solving the minimization of (5)

$$\begin{aligned} &\|\mathbf{J}^{(k)} \delta \mathbf{x} - \mathbf{r}^{(k)}\|^2 \\ &= \left\| \mathbf{Q} \begin{bmatrix} \mathbf{R} \\ \mathbf{0} \end{bmatrix} \delta \mathbf{x} - \mathbf{r}^{(k)} \right\|^2 \end{aligned}$$

$$\begin{aligned} &= \left\| \begin{bmatrix} \mathbf{R} \\ \mathbf{0} \end{bmatrix} \delta \mathbf{x} - \mathbf{Q}^\top \mathbf{r}^{(k)} \right\|^2 = \left\| \begin{bmatrix} \mathbf{R} \\ \mathbf{0} \end{bmatrix} \delta \mathbf{x} - \begin{bmatrix} \mathbf{d} \\ \boldsymbol{\epsilon} \end{bmatrix} \right\|^2 \\ &= \|\mathbf{R} \delta \mathbf{x} - \mathbf{d}\|^2 + \|\boldsymbol{\epsilon}\|^2 \end{aligned} \quad (7)$$

where $\mathbf{R}$ is the so-called *square root information matrix* [17]. Regardless of $\|\boldsymbol{\epsilon}\|^2$ a constant, we have: $\delta \mathbf{x}^{(k)} = \mathbf{R}^{-1} \mathbf{d}$, which can be found via back substitution since $\mathbf{R}$ is upper-triangular. Then, the state estimate is updated as: $\hat{\mathbf{x}}^{(k+1)} = \hat{\mathbf{x}}^{(k)} + \delta \mathbf{x}^{(k)}$.

When new measurements are included as the SLAM goes on, ideally we should recompute (5) and perform factorization (7) from scratch. However, such a batch operation has already been demonstrated to be costly in practice, even after making use of the sparsity of the matrices involved [18], [19]. To save the computation, we can incrementally update (7) using QR factorization with the new measurements, however, based on the previously computed matrices, such as in [20]. Similar to (5), if we have term $\|\mathbf{J}_n \delta \mathbf{x} - \mathbf{r}_n\|^2$ corresponding to the new measurements, the cost function in (7) is updated as²

$$\begin{aligned} &\min_{\delta \mathbf{x}} \|\mathbf{R} \delta \mathbf{x} - \mathbf{d}\|^2 + \|\mathbf{J}_n \delta \mathbf{x} - \mathbf{r}_n\|^2 \\ &\Leftrightarrow \min_{\delta \mathbf{x}} \left\| \begin{bmatrix} \mathbf{R} \\ \mathbf{J}_n \end{bmatrix} \delta \mathbf{x} - \begin{bmatrix} \mathbf{d} \\ \mathbf{r}_n \end{bmatrix} \right\|^2. \end{aligned} \quad (8)$$

Then, we perform QR factorization as follows:

$$\begin{bmatrix} \mathbf{R} \\ \mathbf{J}_n \end{bmatrix} = \mathbf{Q} \begin{bmatrix} \mathbf{R}^\oplus \\ \mathbf{0} \end{bmatrix}, \quad \begin{bmatrix} \mathbf{d}^\oplus \\ \boldsymbol{\epsilon} \end{bmatrix} = \mathbf{Q}^\top \begin{bmatrix} \mathbf{d} \\ \mathbf{r}_n \end{bmatrix} \quad (9)$$

which can be achieved *in-place* for efficiency (e.g., by Givens rotations [51]). Specifically, the superscript $\oplus$ means the result is optimal up to the linearization errors. As a result, we have the updated batch problem

$$\min_{\delta \mathbf{x}} \|\mathbf{R}^\oplus \delta \mathbf{x} - \mathbf{d}^\oplus\|^2. \quad (10)$$

Though the recomputation is avoided by doing incremental update, the efficiency of a batch estimator heavily depends on the computational complexity of (9). Note that once the loop-closure measurements are included, the corresponding $\mathbf{J}_n$ has nonzero blocks of the earlier mapped features and the current pose of the sensing platform, respectively. Thus, in the worst case the computational complexity of QR will be quadratic to the dimension of $\mathbf{x}$. In particular, the sparsity of $\mathbf{R}$ dominates the efficiency in that case, and the approaches such as variable reordering (e.g., [20] and [43]) and sparsification (e.g., [52] and [53]) are used to keep $\mathbf{R}$ sparse. However, as the size of the graph (i.e., the total number of $\mathcal{X}$ and $\mathcal{Z}$) grows, the computational cost on those approaches will also grow, and therefore impact the efficiency of the estimator adversely [53]. As a result, the batch solutions may still be too computationally expensive to implement for online large-scale SLAM applications.

²If new nodes are added with those edges, $\mathbf{R}$ should first be expanded by the appropriate number of *empty* columns, such that $\dim(\mathbf{R}) = \dim(\mathbf{J}_n)$.

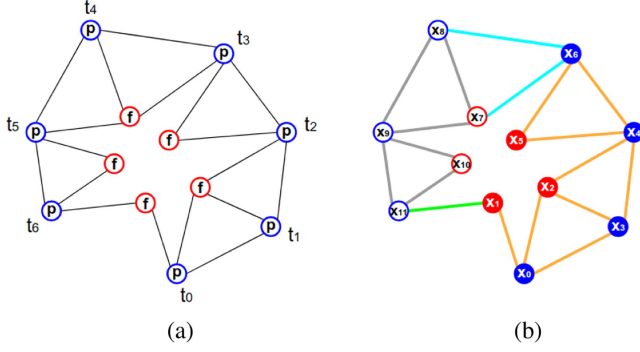


Fig. 1. Example of SLAM (using seven poses and five features): (a) A graph formulation of batch SLAM problem. (b) For our parallel estimator design we index the nodes from 0 to 11, and thus we have $\mathcal{I}_W = \{7, 8, 9, 10, 11\}$, $\mathcal{I}_M = \{0, 1, 2, 3, 4, 5, 6\}$, and $\mathcal{I}_{WM} = \{1, 6\}$. The edges in $\mathcal{Z}_W^F$ are colored gray, and the edges in $\mathcal{Z}_M^B$ are colored orange. $\mathcal{Z}_W^B$ includes both $\mathcal{Z}_W^F$ and the edges in cyan and green that connect sliding window ($\mathcal{X}_W$) and map ($\mathcal{X}_M$), where $\mathbf{z}_{11,1}$ is the loop-closure edge. (a) Graph of SLAM. (b) Partition scheme.

IV. CONSISTENT PARALLEL SLAM ESTIMATOR

In this section, we explain our parallel estimator designed for solving the VI-SLAM problem consistently in real-time, which comprises of two threads: 1) the front-end is used only for providing real-time estimates of the most recent states; 2) the back-end running in parallel will optimize the entire graph by fusing loop-closure measurements online.

For the ensuing presentation, we first give the following definitions (see Fig. 1 for an illustrative example).

- 1) The sliding window in the front-end comprises a constant number of most recent poses and the observed features (excluding the ones involved in the loop closure), and the corresponding nodes are $\mathcal{X}_W = \{\mathbf{x}_i\}_{i \in \mathcal{I}_W}$ with $\mathcal{I}_W \subseteq \mathcal{I}$.
- 2) Those nodes outside the current sliding window form the map: $\mathcal{X}_M = \{\mathbf{x}_i\}_{i \in \mathcal{I}_M}$ with $\mathcal{I}_M = \mathcal{I} \setminus \mathcal{I}_W$.
- 3) For the front-end, we have the edges between the nodes in the current sliding window, $\mathcal{Z}_W^F = \{\mathbf{z}_{ij}\}_{(i,j) \in \mathcal{I}_W}$.
- 4) For the back-end, we denote the edges within the map, $\mathcal{Z}_M^B = \{\mathbf{z}_{ij}\}_{(i,j) \in \mathcal{I}_M}$, and the edges in the sliding window, $\mathcal{Z}_W^B = \mathcal{Z} \setminus \mathcal{Z}_M^B$; while part of the nodes in the map with indexes $\mathcal{I}_{WM}$, that are connected with the nodes in the sliding window, are denoted as $\mathcal{X}_{WM} = \{\mathbf{x}_i\}_{i \in \mathcal{I}_{WM}}$.

A. Front-End

In our design, the front-end runs a *standalone* sliding-window VIO to provide the MAP estimate of $\mathcal{X}_W$, given the measurements $\mathcal{Z}_W^F$ and a prior $p(\mathcal{X}_W^0)$ from the marginalization of the states and measurements moving out of the sliding window. Note that relocalization is *not* initiated by the front-end when loop closure is detected, which is deferred to the back-end. Therefore, without loss of generality, we consider the front-end as a functor which takes $\{\mathcal{Z}_W^F, p(\mathcal{X}_W^0)\}$ as input and outputs consistent estimate $\hat{\mathbf{x}}_W$ in real-time.

B. Back-End

The back-end is running in parallel of the front-end, performing mapping and global optimization that is triggered by the loop closure. In our design, the sliding-window states $\mathcal{X}_W$ being estimated by the front-end are *jointly* adjusted in the batch optimization of the back-end, which is distinct from the other parallel estimators in the literature. This requires back-end to be able to track high-rate front-end's outputs, which is challenging given intrinsically low-rate back-end. To this end, we propose an *asynchronous* solution to integrate this critical high-rate tracking with the low-rate batch optimization.

1) *Functional Decoupling*: With the partitioning of $\mathbf{x} = [\mathbf{x}_M^\top \mathbf{x}_W^\top]^\top$ (which corresponds to all the states $\mathcal{X}$) and $\mathcal{Z} = \mathcal{Z}_M^B \cup \mathcal{Z}_W^B$, for the batch MLE (4) we first linearize the edges in $\mathcal{Z}_W^B$ at current estimate $\hat{\mathbf{x}} = [\hat{\mathbf{x}}_M^\top \hat{\mathbf{x}}_W^\top]^\top$ (i.e., $\hat{\mathbf{x}}^{(0)}$), where $\hat{\mathbf{x}}_W$ is obtained from the front-end

$$\begin{aligned}
 c &\simeq \sum_{\mathbf{z}_{ij} \in \mathcal{Z}_M^B} \|\mathbf{z}_{ij} - \mathbf{h}(\mathbf{x}_i, \mathbf{x}_j)\|_{\Lambda_{ij}}^2 + \sum_{\mathbf{z}_{ij} \in \mathcal{Z}_W^B} \|\mathbf{H}_{ij} \delta \mathbf{x} - \mathbf{e}_{ij}\|_{\Lambda_{ij}}^2 \\
 &= \sum_{\mathbf{z}_{ij} \in \mathcal{Z}_M^B} \|\mathbf{z}_{ij} - \mathbf{h}(\mathbf{x}_i, \mathbf{x}_j)\|_{\Lambda_{ij}}^2 + \|\mathbf{J}_W^B \delta \mathbf{x} - \mathbf{r}_W\|^2 \\
 &= \sum_{\mathbf{z}_{ij} \in \mathcal{Z}_M^B} \|\mathbf{z}_{ij} - \mathbf{h}(\mathbf{x}_i, \mathbf{x}_j)\|_{\Lambda_{ij}}^2 + \|\mathbf{J}_{WM} \delta \mathbf{x}_{WM} \\
 &\quad + \mathbf{J}_W \delta \mathbf{x}_W - \mathbf{r}_W\|^2 \\
 &=: c_M + c_W
 \end{aligned} \tag{11}$$

where we have already used the fact that per the definitions in Section IV $\mathcal{Z}_W^B$ is correlated to both $\mathbf{x}_M$ (i.e., $\mathbf{x}_{WM}$) and $\mathbf{x}_W$, and thus $\mathbf{J}_W^B$ is in the form of $\mathbf{J}_W^B := [[\mathbf{0} \ \mathbf{J}_{WM}] \ \mathbf{J}_W]$. Note also that c_M is only related to $\mathbf{x}_M$.

In minimizing (11) (i.e., $\delta \mathbf{x}^* = \arg \min_{\delta \mathbf{x}} c$), we first solve $\arg \min_{\delta \mathbf{x}_M} c_M$ for the optimal $\delta \mathbf{x}_M^*$ (and hence, $\delta \mathbf{x}_{WM}^*$ which is part of $\delta \mathbf{x}_M^*$), and then substitute $\delta \mathbf{x}_{WM}^*$ into the linear least square of c_W to obtain the optimal update $\delta \mathbf{x}_W^*$ for the front-end estimates. If new edges are added into the graph during the minimization of c_M , their corresponding cost terms will be incorporated into c_W . Note that $\delta \mathbf{x}_M^*$ would not be optimal if not using the latest loop-closure edges. As such, the optimal $\delta \mathbf{x}^*$ ideally cannot be attained until all the loop-closure edges are absorbed into $\mathcal{Z}_M^B$ from $\mathcal{Z}_W^B$, which however inevitably leads to delayed global optimization. Clearly, this delay could be detrimental for real-time SLAM applications, where loop-closing adjustment is needed to correct navigation drift timely. To overcome this limitation, here we present an asynchronous method for triggering global optimization upon the arrival of loop-closure edges. In particular, we leverage the incremental update strategy to insert new incoming edges (including new loop-closure edges) into the global optimization in real-time.

2) *Parallel Optimization*: Assuming that a loop closure is detected and the corresponding edges are added into $\mathcal{Z}$ (more precisely, $\mathcal{Z}_W^B$), which triggers global optimization. To solve for $\delta \mathbf{x}^*$, we perform the following steps.

a) *Factorize c_W* : Note that $\mathbf{J}_W$ becomes a tall matrix after including the Jacobians of the loop-closure edges. Thus, we refactor c_W based on a QR factorization on $\mathbf{J}_W$

$$\begin{aligned}
 c_W &= \|\mathbf{J}_W \delta \mathbf{x}_W + \mathbf{J}_{WM} \delta \mathbf{x}_{WM} - \mathbf{r}_W\|^2 \\
 &= \left\| \mathbf{Q} \begin{bmatrix} \mathbf{R}_W \\ \mathbf{0} \end{bmatrix} \delta \mathbf{x}_W + \mathbf{J}_{WM} \delta \mathbf{x}_{WM} - \mathbf{r}_W \right\|^2 \\
 &= \left\| \begin{bmatrix} \mathbf{R}_W \\ \mathbf{0} \end{bmatrix} \delta \mathbf{x}_W + \mathbf{Q}^\top \mathbf{J}_{WM} \delta \mathbf{x}_{WM} - \mathbf{Q}^\top \mathbf{r}_W \right\|^2 \\
 &= \left\| \begin{bmatrix} \mathbf{R}_W \\ \mathbf{0} \end{bmatrix} \delta \mathbf{x}_W + \begin{bmatrix} \mathbf{R}_{WM} \\ \mathbf{H}_{WM} \end{bmatrix} \delta \mathbf{x}_{WM} - \begin{bmatrix} \mathbf{r}_{WM} \\ \mathbf{e}_{WM} \end{bmatrix} \right\|^2 \\
 &= \|\mathbf{R}_W \delta \mathbf{x}_W + \mathbf{R}_{WM} \delta \mathbf{x}_{WM} - \mathbf{r}_{WM}\|^2 \\
 &\quad + \|\mathbf{H}_{WM} \delta \mathbf{x}_{WM} - \mathbf{e}_{WM}\|^2 \\
 &=: \bar{c}_W + \underline{c}_W
 \end{aligned} \tag{12}$$

where $\underline{c}_W$ is only about $\delta \mathbf{x}_{WM}$ and will be used as the prior to solve for $\delta \mathbf{x}_M^*$; while $\bar{c}_W$ keeps the *correlation* between $\mathbf{x}_W$ and $\mathbf{x}_M$, which will be used to absorb any new incoming measurements in real-time before getting the result of $\delta \mathbf{x}_M^*$.

b) *Solve for $\hat{\mathbf{x}}_M^*$ and thus $\delta \mathbf{x}_{WM}^*$* : We deliberately augment c_M with $\underline{c}_W$ to form a novel MAP estimation problem with respect to $\delta \mathbf{x}_M$ only [see (11)]

$$\begin{aligned}
 \delta \mathbf{x}_M^* &= \arg \min_{\delta \mathbf{x}_M} (c_M + \underline{c}_W) \\
 &= \arg \min_{\delta \mathbf{x}_M} \left(\sum_{\mathbf{z}_{ij} \in \mathcal{Z}_M^B} \|\mathbf{z}_{ij} - \mathbf{h}(\mathbf{x}_i, \mathbf{x}_j)\|_{\Lambda_{ij}}^2 \right. \\
 &\quad \left. + \|\mathbf{H}_{WM} \delta \mathbf{x}_{WM} - \mathbf{e}_{WM}\|^2 \right)
 \end{aligned} \tag{13}$$

which can be cast as a BA problem and solved iteratively. In particular, for estimation consistency [54], at k th iteration we minimize the above cost function as follows:

$$\begin{aligned}
 c_M(\hat{\mathbf{x}}_M^{(k)} + \delta \mathbf{x}_M) &\simeq \|\mathbf{J}_M^{(k)} \delta \mathbf{x}_M - \mathbf{r}_M^{(k)}\|^2 + \|\mathbf{H}_{WM} \delta \mathbf{x}_{WM} \\
 &\quad + \mathbf{H}_{WM} \Delta \mathbf{x}_{WM}^{(k)} - \mathbf{e}_{WM}\|^2 \\
 &= \|\mathbf{J}_M^{(k)} \delta \mathbf{x}_M - \mathbf{r}_M^{(k)}\|^2 + \|\mathbf{H}_{WM} \delta \mathbf{x}_{WM} \\
 &\quad - (\mathbf{e}_{WM} - \mathbf{H}_{WM} \Delta \mathbf{x}_{WM}^{(k)})\|^2 \\
 &=: \|\mathbf{J}_M^{(k)} \delta \mathbf{x}_M - \mathbf{r}^{(k)}\|^2 + \|\mathbf{H}_{WM} \delta \mathbf{x}_{WM} - \mathbf{e}^{(k)}\|^2
 \end{aligned} \tag{14}$$

where $\Delta \mathbf{x}_{WM}^{(k)} := \hat{\mathbf{x}}_{WM}^{(k)} - \hat{\mathbf{x}}_{WM}^{(0)}$. Every iteration we update the estimate of $\mathbf{x}_M$ as: $\hat{\mathbf{x}}_M^{(k+1)} = \hat{\mathbf{x}}_M^{(k)} + \delta \mathbf{x}_M^{(k)}$. Once it converges, we obtain $\hat{\mathbf{x}}_M^*$ (and $\hat{\mathbf{x}}_{WM}^*$), and thus, $\delta \mathbf{x}_{WM}^* = \hat{\mathbf{x}}_{WM}^* - \hat{\mathbf{x}}_{WM}^{(0)}$.

c) *Update $\bar{c}_W$* : Depending on the size of graph, Step-b) requires time to be finished, while new measurements are still arriving in real-time. In order to process them without delay,

here we perform incremental update [see (9)] on $\bar{c}_W$ with their corresponding cost term³ $c_n := \|\mathbf{J}_n \delta \mathbf{x} - \mathbf{r}_n\|^2$, as

$$\begin{aligned}
 \bar{c}_W &= \|\mathbf{R}_W \delta \mathbf{x}_W + \mathbf{R}_{WM} \delta \mathbf{x}_{WM} - \mathbf{r}_{WM}\|^2 + \|\mathbf{J}_n \delta \mathbf{x} - \mathbf{r}_n\|^2 \\
 &= \left\| \begin{bmatrix} \mathbf{R}_W \\ \mathbf{J}_{Wn} \end{bmatrix} \delta \mathbf{x}_W + \begin{bmatrix} \mathbf{R}_{WM} \\ \mathbf{J}_{WMn} \end{bmatrix} \delta \mathbf{x}_{WM} - \begin{bmatrix} \mathbf{r}_{WM} \\ \mathbf{r}_n \end{bmatrix} \right\|^2 \\
 &= \left\| \mathbf{Q} \begin{bmatrix} \mathbf{R}_W^\oplus \\ \mathbf{0} \end{bmatrix} \delta \mathbf{x}_W + \begin{bmatrix} \mathbf{R}_{WM} \\ \mathbf{J}_{WMn} \end{bmatrix} \delta \mathbf{x}_{WM} - \begin{bmatrix} \mathbf{r}_{WM} \\ \mathbf{r}_n \end{bmatrix} \right\|^2 \\
 &= \left\| \begin{bmatrix} \mathbf{R}_W^\oplus \\ \mathbf{0} \end{bmatrix} \delta \mathbf{x}_W + \mathbf{Q}^\top \begin{bmatrix} \mathbf{R}_{WM} \\ \mathbf{J}_{WMn} \end{bmatrix} \delta \mathbf{x}_{WM} - \mathbf{Q}^\top \begin{bmatrix} \mathbf{r}_{WM} \\ \mathbf{r}_n \end{bmatrix} \right\|^2 \\
 &= \left\| \begin{bmatrix} \mathbf{R}_W^\oplus \\ \mathbf{0} \end{bmatrix} \delta \mathbf{x}_W + \begin{bmatrix} \mathbf{R}_{WM}^\oplus \\ \mathbf{H}_n \end{bmatrix} \delta \mathbf{x}_{WM} - \begin{bmatrix} \mathbf{r}_{WM}^\oplus \\ \mathbf{e}_n \end{bmatrix} \right\|^2 \\
 &= \|\mathbf{R}_W^\oplus \delta \mathbf{x}_W + \mathbf{R}_{WM}^\oplus \delta \mathbf{x}_{WM} - \mathbf{r}_{WM}^\oplus\|^2 \\
 &\quad + \|\mathbf{H}_n \delta \mathbf{x}_{WM} - \mathbf{e}_n\|^2
 \end{aligned} \tag{15}$$

$$\simeq \|\mathbf{R}_W^\oplus \delta \mathbf{x}_W + \mathbf{R}_{WM}^\oplus \delta \mathbf{x}_{WM} - \mathbf{r}_{WM}^\oplus\|^2 \tag{16}$$

where we have approximated $\bar{c}_W$ with only the first term in (15). Ideally, the other term of $\delta \mathbf{x}_{WM}$ -only in (15) should be combined with c_M as in (13) for processing, which, however will slow down the global optimization as the ongoing Step-b) needs to be suspended and re-run. To parallelize the processes in Step-b) and Step-c), we drop this term according to [55, Lemma 2], that, as a remedy, saves the computation cost while still keeping the estimation consistency. Note that the sliding-window marginalization if necessary can be further performed based on (16) thanks to the square-root formulation, for which we will present a use case in Section V-B.

d) *Solve for $\hat{\mathbf{x}}_W^*$* : Once Step-b) gets finished, we can use current $\bar{c}_W$ to solve for $\delta \mathbf{x}_W^*$. As $\bar{c}_W$ is linearized at $\hat{\mathbf{x}}_{WM}^{(0)}$ [see (12)], we substitute $\delta \mathbf{x}_{WM}^*$ into (16), such that

$$\begin{aligned}
 \bar{c}_W &= \|\mathbf{R}_W^\oplus \delta \mathbf{x}_W + \mathbf{R}_{WM}^\oplus \delta \mathbf{x}_{WM}^* - \mathbf{r}_{WM}^\oplus\|^2 \\
 &= \|\mathbf{R}_W^\oplus \delta \mathbf{x}_W - \mathbf{r}_W^*\|^2
 \end{aligned} \tag{17}$$

$$\Rightarrow \delta \mathbf{x}_W^* = \arg \min_{\delta \mathbf{x}_W} \bar{c}_W = \mathbf{R}_W^{\oplus -1} \mathbf{r}_W^* \tag{18}$$

where $\mathbf{r}_W^* := \mathbf{r}_{WM}^\oplus - \mathbf{R}_{WM}^\oplus \delta \mathbf{x}_{WM}^*$. As $\mathbf{R}_W^\oplus$ is already upper-triangular, (18) can be efficiently solved via back substitution. As a result, we can immediately have the optimized front-end state estimate with loop closure: $\hat{\mathbf{x}}_W^* = \hat{\mathbf{x}}_W + \delta \mathbf{x}_W^*$.

C. Remarks

It is interesting to examine the computational complexity of the total operations needed in our proposed global optimization. Assuming $|\mathcal{X}_W| = n$ and $|\mathcal{Z}_W^B| = m$ when detecting the loop

³Here, $\mathbf{J}_n := [\mathbf{0} \ \mathbf{J}_{WMn}] \mathbf{J}_{Wn}$. As new states will be added into $\mathbf{x}_W$, we need to expand $\mathbf{R}_W$ with the corresponding amount of empty columns. If c_n include new *loop-closure* edges, then $\mathbf{J}_{WMn} \neq \mathbf{0}$ which corresponds to the new reobserved features that are added into $\mathbf{x}_{WM}$. In such a case, we need to expand $\mathbf{R}_{WM}$ with the appropriate amount of empty columns also.

closure and the number of the reobserved features is l , the computational complexity of constructing c_W is $O(m + l)$. QR factorization for (12) can be performed in-place using Givens rotations [51], therefore the computational complexity of Step-a) is $O(m(n + l)^2)$. For Step-b), the computational complexity of BA depends on the solver used. However, this is not critical as we can run it in a separate thread at its own speed. Instead, we care more about the speed of Step-c), because it must be running in real time. Assuming there are p new measurements arriving at each timestep, the computational complexity of QR factorization in (16) is $O(p(n + l)^2)$ if using Givens rotations. In particular, the sliding-window marginalization can be performed together with a single QR factorization [56] so that the computational complexity of (16) becomes $O((n + p)(n + l)^2)$, where we assume the number of states in a sliding window is always n . As typically $m \gg n > p > l$ in practice, the real-time performances of Step-a) and Step-c) are both achievable. Finally, in Step-d) the computational complexity of computing the residual and back-substitution is $O(n^2)$.

If sliding-window marginalization is performed along with (16), then the marginalized states will not be updated by the ongoing global optimization. However, as those states will be moved into $\mathcal{X}_M$, their estimates are able to be optimized as long as: 1) there will be loop closures in the future; or 2) we perform a full BA including all the states and measurements at the end. As the state estimates from the front-end are *only* used as the linearization points by the back-end, different state parameterizations can be used by front-end and back-end. For example, the robocentric formulation [16] can be used in the front-end to benefit estimation consistency, while a classical world-centric formulation can be used in the back-end in favor of global mapping. Accordingly, we can also model the measurements differently regarding the parameterization used. For example, in order for better linearity we can use inverse-depth model [57] in the front-end to initialize the features, then the standard Euclidean coordinates and the corresponding reprojection model can be applied to the same features in the back-end to reduce the size of the BA problem and save the computational cost of batch optimization.

V. HYBRID DESIGN OF PARALLEL VI-SLAM

In this section, we are to present in detail a hybrid design of the parallel SLAM estimator for monocular VI systems, which comprises a *robocentric* front-end and a *world-centric* back-end for achieving consistent real-time estimation, as an application of our proposed framework in Section IV.

A. Front-End (Robocentric VIO)

The front-end is developed based on our recent work of R-VIO2 [56] which realizes a square-root robocentric estimator for real-time 3-D motion tracking. Here, we augment its function to estimate a local map so that we can add these features in the back-end for global mapping. As we have seen in [16], to keep estimation consistency the robocentric estimator uses local parameterization for the features. For a sliding-window estimator we should change the anchor pose of the local map

once it is marginalized. However, one concern on doing this is when transforming a local map from one pose to another the linearization errors in the pose estimates are propagated into the estimates of features. Instead of that, we rebase those local features to the global frame after the initial update and use such estimates in the future estimation (i.e., first-estimates Jacobian (FEJ) principle [58]). Especially, their measurements will only be used for updating the state vector but not for the square root information matrix, so as to keep the observability properties of robocentric VIO [16]. In what follows, we will briefly explain how our front-end works, while please refer to the corresponding appendices for the details of the algorithm.

1) *State Vector*: At timestep k , the original state vector of R-VIO2 is augmented in the front with the global positions of the currently observed features⁴

$$\mathbf{x}_{VIO} =: \mathbf{x}_k = \begin{bmatrix} \mathbf{x}_{\mathcal{F}_k}^\top & \mathbf{x}_{\mathcal{G}_k}^\top & \mathbf{x}_{\mathcal{P}_k}^\top & \mathbf{x}_{\mathcal{W}_k}^\top \end{bmatrix}^\top \quad (19)$$

where $\mathbf{x}_{\mathcal{F}_k} = [{}^G\mathbf{p}_{f_1}^\top \cdots {}^G\mathbf{p}_{f_{L-1}}^\top {}^G\mathbf{p}_{f_L}^\top]^\top$ consists of the 3-D positions of L features expressed in the global frame $\{G\}$; $\mathbf{x}_{\mathcal{G}_k} = [{}^k_G\bar{q}^\top {}^{R_k}\mathbf{p}_G^\top {}^{R_k}\mathbf{g}^\top]^\top$ with ${}^k_G\bar{q}$ (4×1 quaternion [60]) and ${}^{R_k}\mathbf{p}_G$ (3×1 Euclidean coordinates) the orientation and position of the global frame $\{G\}$ with respect to the robocentric frame of reference $\{R_k\}$ (i.e., coincident with IMU frame $\{I\}$ at timestamp t_k) and ${}^{R_k}\mathbf{g}$ a unit vector of local gravity in $\{R_k\}$; $\mathbf{x}_{\mathcal{P}_k} = [{}^C_I\bar{q}^\top {}^C_I\mathbf{p}_I^\top t_d]^\top$ comprises of online calibration parameters with ${}^C_I\bar{q}$ and ${}^C_I\mathbf{p}_I$ the rotation and translation between camera frame $\{C\}$ and IMU frame $\{I\}$, and t_d the offset between the timestamps of a pair of camera and IMU measurements received at timestep k . The sliding window $\mathbf{x}_{\mathcal{W}_k}$ consists of relative IMU poses between recent N timesteps, where the last one additionally keeps the IMU velocity ${}^{I_k}\mathbf{v}$, biases $\mathbf{b}_{g_k}$ and $\mathbf{b}_{a_k}$ expressed in $\{I_k\}$.

2) *Sensor Factors*: For IMU and camera measurements, we build IMU term $\mathcal{C}_{IMU}$, and three camera terms according to the feature types: a) local feature initialization $\mathcal{C}_{f,init}$ by the features that can be tracked across current sliding window; b) multiview pose constraint $\mathcal{C}_{f,pose}$ by any features that can be continuously tracked over at least two timesteps; and c) global reprojection constraint $\mathcal{C}_{f,repj}$ by those reobserved features in $\mathbf{x}_{\mathcal{F}}$ (see Appendices A and B for the derivations), which convey the information of respective measurements.

3) *Estimation*: An MAP estimator is formed by combining all the information factors within the current sliding window to solve for the state update, in a general form, as

$$\begin{aligned} \delta \hat{\mathbf{x}}_{VIO}^* &= \arg \min_{\delta \mathbf{x}} \mathcal{C}_{VIO} \\ &= \arg \min_{\delta \mathbf{x}} (\mathcal{C}_{Prior} + \mathcal{C}_{IMU} + \mathcal{C}_f) \end{aligned} \quad (20)$$

where $\mathcal{C}_{Prior}$ is the prior factor containing all the information of the states up to the current timestep, and $\mathcal{C}_f$ collects all the available factors of features. The state should then be updated as:

⁴In what follows, left superscript if shown denotes the frame of reference with respect to which a vector is expressed. $\mathbf{I}_n$ and $\mathbf{0}_n$ are the $n \times n$ identity and zero matrices, respectively. For the error quaternion we quantify it using its associated 3-D error angle $\delta\theta$ (i.e., $\delta\bar{q} \simeq [\frac{1}{2}\delta\theta^\top \ 1]^\top$) [59].

$\hat{\mathbf{x}}_{\text{VIO}}^* = \hat{\mathbf{x}}_{\text{VIO}} \boxplus \delta \hat{\mathbf{x}}_{\text{VIO}}^*$.⁵ Note that, specific care is taken in our robocentric estimator to incorporate local mapping, for which we have the details shown in Appendices C and D.

At each timestep, we send raw measurements used, the updated state estimate $\hat{\mathbf{x}}_{\text{VIO}}$, and the corresponding information factor $\mathcal{C}_{\text{VIO}}$ to the back-end for further processing.

B. Back-End (Asynchronous BA)

The back-end of our system uses all the sensor measurements and state estimates (as the linearization points) passed from the front-end to perform BA-based optimization. Unlike some well-known visual-inertial mapping systems where the back-end is mainly used as an *offline* tool for building large-scale maps (e.g., maplab [61]), we perform *online* SLAM with loop closures aiming to correct the front-end drift in time. Also, different from the state-of-the-art visual(-inertial) SLAM algorithms [40], [41], [42] where only keyframes (or a pose graph) are optimized in a loosely-coupled way to reduce the computational complexity of BA, we jointly estimate the entire trajectory and observed features up to current time. To apply timely loop-closure correction to both front-end and back-end, in a tightly-coupled and consistent manner, we propose the following asynchronous optimization scheme.

As evident from Section IV, the back-end SLAM graph has two types of nodes, sensor poses and feature positions

$$\mathbf{x}_I = \begin{bmatrix} {}^I_G \bar{\mathbf{q}}^\top & {}^G \mathbf{p}_I^\top & {}^G \mathbf{v}_I^\top & \mathbf{b}_g^\top & \mathbf{b}_a^\top \end{bmatrix}^\top \quad (21)$$

$$\mathbf{x}_f =: {}^G \mathbf{p}_f = [x \ y \ z]^\top. \quad (22)$$

Note that the pose state is augmented by the global velocity, ${}^G \mathbf{v}_I$, and IMU biases, $\mathbf{b}_g$ and $\mathbf{b}_a$, while the feature position is expressed with respect to the global frame $\{G\}$. There are two types of graph edges also, IMU motions and feature observations. Here, the former uses IMU preintegration model proposed in [62], while the latter is derived based on (53).

Specifically, to describe relative motion between two IMU poses, i and j , we have the following equations based on the preintegration of the corresponding IMU measurements:⁶

$${}^j_G \bar{\mathbf{q}} = {}^j_i \bar{\mathbf{q}} \otimes {}^i_G \bar{\mathbf{q}} \quad (23)$$

$${}^G \mathbf{p}_{I_j} = {}^G \mathbf{p}_{I_i} + {}^G \mathbf{v}_{I_i} \Delta t_{ij} - \frac{1}{2} {}^G \mathbf{g} \Delta t_{ij}^2 + {}^i_G \mathbf{C}_{\bar{\mathbf{q}}}^\top \boldsymbol{\alpha}_{ij} \quad (24)$$

$${}^G \mathbf{v}_{I_j} = {}^G \mathbf{v}_{I_i} - {}^G \mathbf{g} \Delta t_{ij} + {}^i_G \mathbf{C}_{\bar{\mathbf{q}}}^\top \boldsymbol{\beta}_{ij} \quad (25)$$

where $\Delta t_{ij} = t_j - t_i$, ${}^j_i \bar{\mathbf{q}}$, $\boldsymbol{\alpha}_{ij}$, and $\boldsymbol{\beta}_{ij}$ are IMU preintegrated measurements whose values are calculated by assuming IMU biases $\mathbf{b}_g$ and $\mathbf{b}_a$ are constant in $[t_i, t_j]$ [62]. Then, the residual $\mathbf{e}_{I_{ij}} = [\delta_i^j \boldsymbol{\theta}^\top \ \delta \boldsymbol{\alpha}_{ij}^\top \ \delta \boldsymbol{\beta}_{ij}^\top \ \delta \mathbf{b}_g^\top \ \delta \mathbf{b}_a^\top]^\top$ is given by

$$\delta_i^j \boldsymbol{\theta} = 2({}^j_G \bar{\mathbf{q}} \otimes {}^i_G \bar{\mathbf{q}}^{-1} \otimes {}^j_i \hat{\bar{\mathbf{q}}}^{-1} \otimes \bar{\mathbf{q}}_b)_{1:3} \quad (26)$$

⁵As the update for quaternion is non-Euclidean (using the operator $\otimes$ [59]), we use $\boxplus$ to represent a comprehensive state update.

⁶Note that ${}^b_a \mathbf{C}_{\bar{\mathbf{q}}} \triangleq \mathbf{C}({}_a^b \bar{\mathbf{q}})$ is the 3×3 rotation matrix corresponding to the 4×1 quaternion ${}_a^b \bar{\mathbf{q}}$.

$$\delta \boldsymbol{\alpha}_{ij} = {}^i_G \mathbf{C}_{\bar{\mathbf{q}}} \left({}^G \mathbf{p}_{I_j} - {}^G \mathbf{p}_{I_i} - {}^G \mathbf{v}_{I_i} \Delta t_{ij} + \frac{1}{2} {}^G \mathbf{g} \Delta t_{ij}^2 \right) - \mathbf{J}_\alpha (\mathbf{b}_{g_i} - \hat{\mathbf{b}}_{g_i}) - \mathbf{H}_\alpha (\mathbf{b}_{a_i} - \hat{\mathbf{b}}_{a_i}) - \hat{\boldsymbol{\alpha}}_{ij} \quad (27)$$

$$\delta \boldsymbol{\beta}_{ij} = {}^i_G \mathbf{C}_{\bar{\mathbf{q}}} ({}^G \mathbf{v}_{I_j} - {}^G \mathbf{v}_{I_i} + {}^G \mathbf{g} \Delta t_{ij}) - \mathbf{J}_\beta (\mathbf{b}_{g_i} - \hat{\mathbf{b}}_{g_i}) - \mathbf{H}_\beta (\mathbf{b}_{a_i} - \hat{\mathbf{b}}_{a_i}) - \hat{\boldsymbol{\beta}}_{ij} \quad (28)$$

$$\delta \mathbf{b}_g = \mathbf{b}_{g_j} - \mathbf{b}_{g_i} \quad (29)$$

$$\delta \mathbf{b}_a = \mathbf{b}_{a_j} - \mathbf{b}_{a_i} \quad (30)$$

where ${}^j_i \hat{\bar{\mathbf{q}}}$, $\hat{\boldsymbol{\alpha}}_{ij}$, and $\hat{\boldsymbol{\beta}}_{ij}$ are computed based on the linearization point $\hat{\mathbf{x}}_{I_i}$ (more precisely, $\hat{\mathbf{b}}_{g_i}$ and $\hat{\mathbf{b}}_{a_i}$), with $\mathbf{J}_\alpha$, $\mathbf{H}_\alpha$, $\mathbf{J}_\beta$, and $\mathbf{H}_\beta$ the Jacobians derived for bias compensation; $\bar{\mathbf{q}}_b$ is a unit quaternion corresponding to the change of gyro bias $\mathbf{b}_g$, and $(\cdot)_{1:3}$ is an operator for extracting the top three entries from a quaternion. By linearizing the above residual at $\hat{\mathbf{x}}_{I_i}$ and $\hat{\mathbf{x}}_{I_j}$, we obtain the cost function of IMU edge

$$\mathcal{C}_{I_{ij}} = \|\mathbf{H}_{I_i} \delta \mathbf{x}_{I_i} + \mathbf{H}_{I_j} \delta \mathbf{x}_{I_j} - \mathbf{e}_{I_{ij}}\|_{\boldsymbol{\Sigma}_{I_{ij}}}^2 \quad (31)$$

where $\mathbf{H}_{I_i}$ and $\mathbf{H}_{I_j}$ are the Jacobians of $\mathbf{e}_{I_{ij}}$ with respect to $\mathbf{x}_{I_i}$ and $\mathbf{x}_{I_j}$, respectively, and $\boldsymbol{\Sigma}_{I_{ij}}$ is the residual covariance (please refer to [62] for the details of $\mathbf{H}_{I_i}$, $\mathbf{H}_{I_j}$, and $\boldsymbol{\Sigma}_{I_{ij}}$).

We use perspective projection model (53) for feature edges. The position of feature j in the camera frame i is given as

$$\begin{aligned} {}^C_i \mathbf{p}_{f_j} &= {}^C_i \mathbf{C}_{\bar{\mathbf{q}}} {}^I_i \mathbf{p}_{f_j} + {}^C_i \mathbf{p}_I \\ &= {}^C_i \mathbf{C}_{\bar{\mathbf{q}}} {}^i_G \mathbf{C}_{\bar{\mathbf{q}}} ({}^G \mathbf{p}_{f_j} - {}^G \mathbf{p}_{I_i}) + {}^C_i \mathbf{p}_I. \end{aligned} \quad (32)$$

By linearizing (53) at $\hat{\mathbf{x}}_{f_j}$ (i.e., ${}^G \hat{\mathbf{p}}_{f_j}$) and $\hat{\mathbf{x}}_{I_i}$, we obtain the residual for measurement $\mathbf{z}_{j,i}$ as

$$\mathbf{e}_{f_{ij}} = \mathbf{z}_{j,i} - \hat{\mathbf{z}}_{j,i} \simeq \mathbf{H}_{f_i} \delta \mathbf{x}_{I_i} + \mathbf{H}_{f_j} \delta \mathbf{x}_{f_j} + \mathbf{n}_{\text{im}} \quad (33)$$

with the expressions of $\mathbf{H}_{f_i}$ and $\mathbf{H}_{f_j}$ given in Appendix E. It is easy to get the corresponding cost function, as

$$\mathcal{C}_{f_{ij}} = \|\mathbf{H}_{f_i} \delta \mathbf{x}_{I_i} + \mathbf{H}_{f_j} \delta \mathbf{x}_{f_j} - \mathbf{e}_{f_{ij}}\|_{\boldsymbol{\Sigma}_f}^2. \quad (34)$$

Note that we have used the same estimates to initialize both front-end and back-end for their respective MAP estimation. We do not estimate spatiotemporal calibration parameters in the back-end but use their estimates in front-end as the prior when evaluating the cost functions. Especially, we will handle three cases in practice, as shown in what follows.

1) *No Loop Closure*: If no loop closure is detected, we simply partition the edges into different groups as in Section IV, and the cost function of BA can be expressed as

$$\begin{aligned} \mathcal{C}_{BA} &= \mathcal{C}_W + \mathcal{C}_M \\ &= \sum_{i \in \mathcal{I}_W, j \in \mathcal{I}_W \cup \mathcal{I}_{WM}} (\mathcal{C}_{I_{ij}} + \mathcal{C}_{f_{ij}}) + \sum_{i, j \in \mathcal{I}_M} (\mathcal{C}_{I_{ij}} + \mathcal{C}_{f_{ij}}). \end{aligned} \quad (35)$$

Correspondingly, all the sensor poses and feature points are partitioned into $\mathcal{X}_W$ and $\mathcal{X}_M$. In particular, we will augment $\mathcal{X}_W$ and $\mathcal{C}_W$ once new states and measurements are included, while along with sliding-window marginalization in the front-end we will move the marginalized nodes and edges from $\mathcal{X}_W$ and $\mathcal{C}_W$

to $\mathcal{X}_M$ and $\mathcal{C}_M$. Note that the edges in $\mathcal{C}_W$ are partially connected with the nodes in the map, whereas the edges in $\mathcal{C}_M$ only connect the nodes in the map. We immediately evaluate the edges in $\mathcal{C}_M$ to save time for the batch optimization, while for $\mathcal{C}_W$ we do not evaluate them now because the linearization points used are still being estimated by the front-end, and thus we will keep $\hat{\mathcal{X}}_W$ aligned with the front-end's estimates.

2) *Processing Loop Closure*: Assuming l feature points in the map are reobserved at current timestep k , we thus evaluate $\mathcal{C}_W$ (that will be used to track front-end estimation) with the current estimates of $\mathcal{X}_W$ and $\mathcal{X}_M$. To speed up the processing, here we convert the edges in $\mathcal{C}_W$ into the square-root form to support in-place QR factorization

$$\mathcal{C}_{I_{ij}} = \|\mathbf{J}_{I_i} \delta \mathbf{x}_{I_i} + \mathbf{J}_{I_j} \delta \mathbf{x}_{I_j} - \mathbf{r}_{I_{ij}}\|^2 \quad (36)$$

$$\mathcal{C}_{f_{ij}} = \|\mathbf{J}_{f_i} \delta \mathbf{x}_{I_i} + \mathbf{J}_{f_j} \delta \mathbf{x}_{f_j} - \mathbf{r}_{f_{ij}}\|^2 \quad (37)$$

where $\mathbf{J}_{I_i} = \Sigma_{I_{ij}}^{-1/2} \mathbf{H}_{I_i}$, $\mathbf{J}_{I_j} = \Sigma_{I_{ij}}^{-1/2} \mathbf{H}_{I_j}$, $\mathbf{r}_{I_{ij}} = \Sigma_{I_{ij}}^{-1/2} \mathbf{e}_{I_{ij}}$, and $\mathbf{J}_{f_i} = \Sigma_f^{-1/2} \mathbf{H}_{f_i}$, $\mathbf{J}_{f_j} = \Sigma_f^{-1/2} \mathbf{H}_{f_j}$, $\mathbf{r}_{f_{ij}} = \Sigma_f^{-1/2} \mathbf{e}_{f_{ij}}$.

Assuming there are m features and N sensor poses in the sliding window at timestep k (i.e., same as the front-end), the state vector corresponding to $\mathcal{C}_W$ comprises two parts⁷

$$\mathbf{x}_{W_k} = \begin{bmatrix} \mathbf{x}_{f_{\mathcal{M}_1:\mathcal{M}_m}}^\top & \mathbf{x}_{I_{k-N+1:k}}^\top \end{bmatrix}^\top \quad (38)$$

$$\mathbf{x}_{WM_k} = \begin{bmatrix} \mathbf{x}_{I_{k-N}}^\top & \mathbf{x}_{\mathcal{L}_1:\mathcal{L}_l}^\top \end{bmatrix}^\top \quad (39)$$

where $\mathbf{x}_W$ includes all the states in the current sliding window, and $\mathbf{x}_{WM}$ are the states in the map which are involved in part of edges in $\mathcal{C}_W$, such as l loop-closure edges. Specifically, we do not need the entire $\hat{\mathbf{x}}_M$ but just $\hat{\mathbf{x}}_{WM}$ for computation.

a) *Factorize $\mathcal{C}_W$* : We first follow (12) to evaluate all the Jacobians and residuals in $\mathcal{C}_W$ with $\hat{\mathbf{x}}_W$ and $\hat{\mathbf{x}}_{WM}$, and then stack them together to perform a partial QR factorization

$$\begin{aligned} \mathcal{C}_W &= \|\mathbf{J}_W \delta \mathbf{x}_W + \mathbf{J}_{WM} \delta \mathbf{x}_{WM} - \mathbf{r}_W\|^2 \\ &\stackrel{\text{QR}}{=} \|\mathbf{R}_W \delta \mathbf{x}_W + \mathbf{R}_{WM} \delta \mathbf{x}_{WM} - \mathbf{r}_{WM}\|^2 \\ &\quad + \|\mathbf{H}_{WM} \delta \mathbf{x}_{WM} - \mathbf{e}_{WM}\|^2 \\ &=: \bar{\mathcal{C}}_W + \underline{\mathcal{C}}_W. \end{aligned} \quad (40)$$

b) *Solve for $\hat{\mathbf{x}}_M^*$ and $\delta \mathbf{x}_{WM}^*$* : By using $\underline{\mathcal{C}}_W$ as the prior, we combine it to $\mathcal{C}_W$ to perform batch optimization for $\hat{\mathbf{x}}_M$

$$\begin{aligned} \delta \mathbf{x}_M^* &= \arg \min_{\delta \mathbf{x}_M} (\mathcal{C}_M + \underline{\mathcal{C}}_W) \\ \hat{\mathbf{x}}_M^* &= \hat{\mathbf{x}}_M \boxplus \delta \mathbf{x}_M^*. \end{aligned} \quad (41)$$

In practice, we will use the iterative solver (e.g., Ceres [63]) to automatically evaluate all the edges, as in (13). Though the order of states in $\mathbf{x}_M$ is decided by the solver for efficiency, it will not affect the result. Once the optimization converges,

the optimal state correction $\delta \mathbf{x}_{WM}^*$ is obtained as:⁸ $\delta \mathbf{x}_{WM}^* = \hat{\mathbf{x}}_{WM}^* \boxminus \hat{\mathbf{x}}_{WM}$, based on the results of $\hat{\mathbf{x}}_M^*$ and $\hat{\mathbf{x}}_M$.

c) *Update $\bar{\mathcal{C}}_W$* : Before Step-b) is done, we keep updating $\bar{\mathcal{C}}_W$ with the incoming edges, $\mathcal{C}_n = \|\mathbf{J}_n \delta \mathbf{x} - \mathbf{r}_n\|^2$, as in (16)

$$\begin{aligned} \bar{\mathcal{C}}_W &= \|\mathbf{R}_W \delta \mathbf{x}_W + \mathbf{R}_{WM} \delta \mathbf{x}_{WM} - \mathbf{r}_{WM}\|^2 + \|\mathbf{J}_n \delta \mathbf{x} - \mathbf{r}_n\|^2 \\ &\stackrel{\text{QR}}{=} \|\mathbf{R}_W^\oplus \delta \mathbf{x}_W + \mathbf{R}_{WM}^\oplus \delta \mathbf{x}_{WM} - \mathbf{r}_{WM}^\oplus\|^2 \\ &\quad + \|\mathbf{H}_n \delta \mathbf{x}_{WM} - \mathbf{e}_n\|^2 \\ &\simeq \|\mathbf{R}_W^\oplus \delta \mathbf{x}_W + \mathbf{R}_{WM}^\oplus \delta \mathbf{x}_{WM} - \mathbf{r}_{WM}^\oplus\|^2 \end{aligned} \quad (42)$$

where we drop off $\|\mathbf{H}_n \delta \mathbf{x}_{WM} - \mathbf{e}_n\|^2$ as a consistent remedy for not interrupting the ongoing batch optimization (41). While noticing that this term actually provides a prior of $\mathbf{x}_{WM}$, the iterative strategy used by (41) for $\hat{\mathbf{x}}_M$ (and hence $\hat{\mathbf{x}}_{WM}$) shall mitigate the influence of such loss of prior information.

To be consistent with the front-end, we marginalize sensor poses and feature points that move out of the sliding window from (42). Referring to [56, Lemma 2], we refactor $\mathbf{R}_W^\oplus$, by moving the corresponding columns to those states to its left side, to get $\mathbf{R}_W^\ominus$. Accordingly, for $\hat{\mathbf{x}}_W$ those states are moved to its head to get $\hat{\mathbf{x}}_W^\ominus$. A QR factorization is then performed in-place to make $\mathbf{R}_W^\ominus$ upper-triangular again, while here this process also updates $\mathbf{R}_{WM}^\oplus$ and $\mathbf{r}_{WM}^\oplus$ at the same time

$$\mathbf{R}_W = \mathbf{Q}^\top \mathbf{R}_W^\ominus, \quad \mathbf{R}_{WM} = \mathbf{Q}^\top \mathbf{R}_{WM}^\oplus, \quad \mathbf{r}_{WM} = \mathbf{Q}^\top \mathbf{r}_{WM}^\oplus. \quad (43)$$

Then, we discard the rows corresponding to those states from the top of $\mathbf{R}_W$, $\mathbf{R}_{WM}$ and $\mathbf{r}_{WM}$, and those states themselves from the head of $\hat{\mathbf{x}}_W^\ominus$ to get $\bar{\mathcal{C}}_W$ and $\hat{\mathbf{x}}_W$ similar to (40). Note that, the marginalized measurements (edges) are not discarded but saved, so that once loop closure is done we can use them to immediately *recover* the complete BA problem, as in (35).

d) *Solve for $\hat{\mathbf{x}}_W^*$* : Once getting $\delta \mathbf{x}_{WM}^*$ from Step-b), we solve for $\delta \mathbf{x}_W^*$ by following (17) and (18), where

$$\begin{aligned} \bar{\mathcal{C}}_W &= \|\mathbf{R}_W \delta \mathbf{x}_W + \mathbf{R}_{WM} \delta \mathbf{x}_{WM}^* - \mathbf{r}_{WM}\|^2 \\ &= \|\mathbf{R}_W \delta \mathbf{x}_W - \mathbf{r}_W^*\|^2 \end{aligned} \quad (44)$$

with $\mathbf{r}_W^* = \mathbf{r}_{WM} - \mathbf{R}_{WM} \delta \mathbf{x}_{WM}^*$. Then, $\delta \mathbf{x}_W^*$ is computed as

$$\delta \mathbf{x}_W^* = \arg \min_{\delta \mathbf{x}_W} \bar{\mathcal{C}}_W = \mathbf{R}_W^{-1} \mathbf{r}_W^* \quad (45)$$

where $\mathbf{R}_W$ is of bounded (nearly constant) dimension thanks to the marginalization, which favors efficient back substitution. As a result, the optimal estimate for the sliding window states is given by: $\hat{\mathbf{x}}_W^* = \hat{\mathbf{x}}_W \boxplus \delta \mathbf{x}_W^*$. Together with $\hat{\mathbf{x}}_M^*$ from Step-b), we complete our asynchronous optimization in the back-end.

3) *Correcting the Front-End*: As a key feature of our proposed parallel solution, it is able to provide optimal correction to the front-end (VIO) states. As the parameterizations used by front-end and back-end may be different, the correction $\delta \mathbf{x}_W^*$ in (45) should not be directly applied to the state vector of the front-end. Also, for consistency purpose, the information of the estimator should be changed along with the change of

⁷For the sake of introduction, we define two index sets, $\mathcal{L}$ and $\mathcal{M}$, of size l and m , respectively, for the corresponding feature points in $\mathcal{C}_W$.

⁸Operator $\boxminus$ is contrary to $\boxplus$, where for quaternion $\delta \bar{q} \simeq [\frac{1}{2} \delta \boldsymbol{\theta}^\top \quad 1]^\top$ [59], and we can instead use 3-D $\delta \boldsymbol{\theta}$ to model quaternion correction.

Algorithm 1: Consistent Parallel VI-SLAM.
Input: Camera images and IMU measurements.

Front-end (Thread 1):

 Robocentric VIO outputs $\hat{\mathbf{x}}_{VIO}$, $\mathcal{C}_{VIO}$, and synchronized features/IMU measurements, at every image timestep.

Back-end:

Asynchronous BA outputs optimized map and trajectory.

Thread 2 (eternal):
if no loop closure then

 ① Build $\mathcal{C}_{BA}$ using the front-end's output (see (35)).

else if found loop closure then

 ② Factorize $\mathcal{C}_W$ to get $\bar{\mathcal{C}}_W$ and $\mathcal{C}_W$ (see (40)), and initiate **Thread 3** if none is running.

while ④ is not done do

 ③ Update $\bar{\mathcal{C}}_W$ using the front-end's output (see (42) and (43)) that is also saved for ① later.

if ④ is done then

 ⑤ Solve for $\hat{\mathbf{x}}_W^*$ based on $\bar{\mathcal{C}}_W$ (see (45)), and correct $\hat{\mathbf{x}}_{VIO}$ and $\mathcal{C}_{VIO}$ (see Section V-B3) for the front-end (i.e., relocalization).

end if
end while
end if
Thread 3 (temporary):

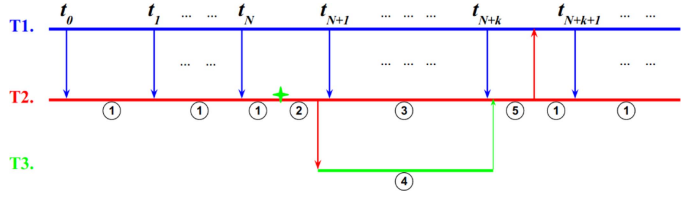
 ④ Solve a batch optimization with $\mathcal{C}_M$ for $\hat{\mathbf{x}}_M^*$, where $\mathcal{C}_W$ from ② is used as the a prior (see (41)).


Fig. 2. Concurrency between the threads (T1–3) in Algorithm 1. t represents image time. The green star indicates the time when loop closure is detected, and the arrows in different colors represent information exchanges between the threads.

TABLE I
PARAMETERS USED FOR SYNTHETIC DATA GENERATION

Parameter	Value
Camera to IMU $\mathcal{C}_{\bar{q}}$	$[0, 0, 0, 1]^\top$
Camera to IMU $\mathcal{C}_{\mathbf{p}_I}$	$[0, 0, 0]^\top$
IMU data rate	100 [Hz]
Gyroscope noise density (σ_g)	$5.0119e^{-4}$ [rad/sec/ $\sqrt{\text{Hz}}$]
Gyroscope bias random walk (σ_{wg})	$3.1623e^{-5}$ [rad/sec ² / $\sqrt{\text{Hz}}$]
Accelerometer noise density (σ_a)	$7.0795e^{-4}$ [m/sec ² / $\sqrt{\text{Hz}}$]
Accelerometer bias random walk (σ_{wa})	$5.6234e^{-5}$ [m/sec ³ / $\sqrt{\text{Hz}}$]
Camera image rate	10 [Hz]
Filed of view	45 [deg]
Lens focal length	484.13161 [pixels]
Image noise level (σ_{im})	1.5 [pixels]
Magnitude of gravity	9.8038 [m/sec ²]

$$R_k \hat{\mathbf{p}}_G^* = \mathbf{C}_{\delta \bar{q}_k^*} (R_k \hat{\mathbf{p}}_G - \delta \mathbf{p}_k^*) \quad (49)$$

$$R_k \hat{\mathbf{g}} = \mathbf{C}_{\delta \bar{q}_k^*} R_k \hat{\mathbf{g}} \quad (50)$$

state estimate. For example, in [50] a prior about the states in the front-end is summarized from the information of the back-end, and incorporated by the front-end along with state update. However, this takes the benefits from both using unified state parameterization in the estimator design and fast information inference of iSAM2 [43], in their implementation. In general, such information summarizing is computationally expensive, and thus cannot guarantee to be real-time if the graph in the back-end becomes large and dense [53]. As such, we do not follow that way, instead propose the following *composition*.

As shown in [16], composition does not alter observability of the robocentric estimator, thus the estimation consistency of our front-end will not be affected. Note that, in our front-end state vector $\mathbf{x}_{VIO}$, $\mathbf{x}_F$ and $\mathbf{x}_G$ are related to the global frame $\{G\}$, whereas $\mathbf{x}_P$ and $\mathbf{x}_V$ are either local or relative states. Although it is able to correct the estimates of all those states, we prefer to update $\hat{\mathbf{x}}_F$ and $\hat{\mathbf{x}}_G$ while leaving $\hat{\mathbf{x}}_P$ and $\hat{\mathbf{x}}_V$ as is their current values, because both local and relative states are independent of the global frame $\{G\}$. For the timestep k , we extract $\hat{\mathbf{x}}_{F_k}^*$ from $\hat{\mathbf{x}}_{f_{M_1:M_m}}^*$, and for $\hat{\mathbf{x}}_{G_k}^*$ we have optimal estimate ${}^k_G \hat{\mathbf{q}}^*$ from $\hat{\mathbf{x}}_{I_k}^*$ [see (38)] directly, while for the others

$$R_k \hat{\mathbf{p}}_G^* = -{}^k_G \mathbf{C}_{\hat{\mathbf{q}}^*}^G \hat{\mathbf{p}}_{I_k}^*, \quad R_k \hat{\mathbf{g}}^* = {}^k_G \mathbf{C}_{\hat{\mathbf{q}}^*}^G \hat{\mathbf{g}} \quad (46)$$

where ${}^G \hat{\mathbf{g}}$ is the vector of global gravity (e.g., $[0, 0, -9.8]^\top$). This is equivalent to performing a composition as below

$${}^G \hat{\mathbf{p}}_{f_i}^* = {}^G \hat{\mathbf{p}}_{f_i} + \delta \mathbf{p}_{f_i}^*, \quad i = 1, \dots, L \quad (47)$$

$${}^k_G \hat{\mathbf{q}}^* = \delta \bar{q}_k^* \otimes {}^k_G \hat{\mathbf{q}} \quad (48)$$

where $\delta \mathbf{p}_{f_i}^*$, $\delta \bar{q}_k^*$, and $\delta \mathbf{p}_k^*$ are the deduced optimal corrections. Then, we can easily find the Jacobian of $\delta \mathbf{x}^*$ with respect to $\delta \mathbf{x}$: $\mathbf{U} = \text{Diag}[\mathbf{I}_{3L} \quad \mathbf{C}_{\delta \bar{q}_k^*} \quad \mathbf{C}_{\delta \bar{q}_k^*} \quad \mathbf{C}_{\delta \bar{q}_k^*} \quad \mathbf{I}_7 \quad \mathbf{I}_{6N+9}]$ with $\mathbf{C}_{\delta \bar{q}_k^*} = {}^k_G \mathbf{C}_{\hat{\mathbf{q}}^*}^k {}^k_G \mathbf{C}_{\hat{\mathbf{q}}^*}^\top$, which is a block-diagonal matrix. We thus follow [56, Lemma 1] to update front-end information matrix, as

$$\mathbf{R}_k \xleftarrow{\text{QR}} \mathbf{R}_k \mathbf{U}^{-1} \quad (51)$$

where we should note that $\mathbf{U}^{-1} = \mathbf{U}^\top$, and QR factorization is used to make the result of $\mathbf{R}_k \mathbf{U}^{-1}$ upper-triangular again. Here, the computation concentrates on the rows corresponding to $\mathbf{x}_G$ in $\mathbf{R}_k$, for $\mathcal{C}_{VIO,k}$, thus the computational complexity of this step is small and bounded, which guarantees efficient *relocalization* for the front-end in a tightly-coupled fashion.

Up to this point, we complete our proposed hybrid parallel VI-SLAM, whose main steps are summarized in Algorithm 1 with the parallelism illustrated by Fig. 2.

VI. SIMULATION RESULTS

In this section, we present the results from the Monte-Carlo simulations to verify our theoretical analysis and demonstrate the performance of the proposed SLAM estimator. The camera and IMU data are generated by simulating a drone flying in a closed-loop corridor. Both camera and IMU parameters use the values provided by the corresponding real sensors, while we assume that $\{C\}$ and $\{I\}$ are coincident with each other (see Table I). In total, we generated 50 trails of visual and inertial data, as well as the map and trajectory groundtruths.

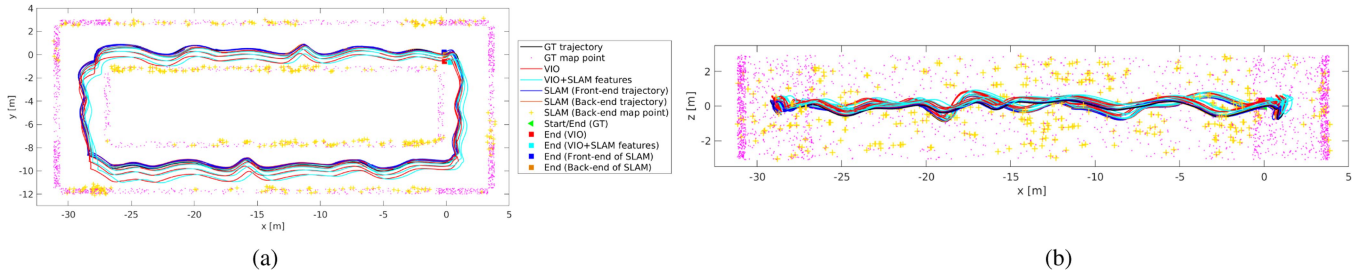


Fig. 3. Estimation result of one of Monte-Carlo simulations, where a local map comprising of eight feature points is estimated by b) VIO with SLAM features and the front-end of c) SLAM. The simulated drone flies five laps by starting and ending at the same point. (a) Overhead plot. (b) Lateral plot.

TABLE II
ERROR STATISTICS BY AVERAGING THE RESULTS OVER 50 MONTE-CARLO SIMULATIONS

Front-end	Number of feature points	VIO	VIO with SLAM features				SLAM			
		—	2	5	8	11	2	5	8	11
Front-end	Orientation RMSE of pose [deg]	1.1803	1.0943	1.0772	1.0485	1.0600	0.1840	0.1790	0.1781	0.1729
	Position RMSE of pose [m]	0.4537	0.4506	0.4479	0.4541	0.4612	0.1804	0.1723	0.1686	0.1700
	Six-DOF Pose NEES	23.9961	21.8695	21.2798	21.2213	21.5664	6.6769	6.3498	6.3218	6.3387
	Position RMSE of map points [m]	—	—	—	—	—	0.0707	0.0725	0.0692	0.0743
Back-end	Orientation RMSE of trajectory [deg]	—	—	—	—	—	0.1229	0.1116	0.1138	0.1129
	Position RMSE of trajectory [m]	—	—	—	—	—	0.0916	0.0892	0.0878	0.0921

Especially, we can easily find loop closures by checking the data association lookup table of each trail. Different settings are tested, including: 1) VIO that estimates a sliding window of the most recent (relative) poses; 2) VIO with SLAM features where a fixed number of feature points that can be observed across more than one sliding window are jointly estimated to provide a local map for VIO; and 3) SLAM that runs with the complete functionality of our estimator, where 2) is used as the front-end, and the back-end as described in Section V-B is running in parallel for mapping and global optimization.

For performance evaluation, we compute root mean squared error (RMSE) and normalized estimation error squared (NEES) [64] for front-end based on its 6 degrees-of-freedom (DOF) pose output to illustrate the accuracy and consistency of the sliding-window estimation. For back-end, we compute RMSE against the groundtruth for estimated maps and trajectories to reveal the optimality of mapping and loop closure. We use Ceres to solve batch optimizations in our back-end, where Levenberg–Marquardt strategy is used in minimizing the cost function, with the maximum number of iterations 50.

Fig. 3 depicts the estimation result of one of the trails, where the groundtruth contains 2376 features in the map and 28901 waypoints along the trajectory. As 25 features are given at each image timestep, we chose a set of number of features {2, 5, 8, 11} to be kept in the VIO state vectors for b) and c) in our comparison. Table II summarizes the results from the error statistics of all 50 trails. First, we should note that after including feature points into VIO estimation the improvements on the pose accuracy are not as obvious as expected, even by increasing the number of feature points. The reason for this is that either a) or b) corresponds to an open-loop system which suffers from the accumulated errors in the state estimates, that is equal to the dead reckoning. Thus, without the awareness of loop closure the drone seems like flying in an infinite corridor. In another hand, the pose NEES is reduced by estimating the

features in VIO, though the ideal value is 6. This also verifies that our proposed method for leveraging local mapping in the robocentric VIO does not impact but improves the consistency of the estimator. Then for c), we should find that by enabling mapping and global optimization all the metrics for the front-end are improved significantly, regardless of the number of feature points. The pose NEES is very close to the ideal value, which means the consistency of our estimator is excellent.

In addition to that, the errors corresponding to the output maps and trajectories of the back-end can be found small in Table II, and their high quality is visualized by Fig. 3. Here, we may have noticed that when we increase the number of feature points in the VIO to 11, the performances of b) and c) show regressions. This phenomenon is related to the feature measurements used for state update [see Appendix B]. As each image provides fixed number of feature measurements, when more of them are used on $\mathcal{C}_{f, \text{repj}}$ the number of $\mathcal{C}_{f, \text{pose}}$ will decrease. As pointed out by [37], the optimality of such hybrid estimator is related to not only the number of SLAM features, but also the lengths of their tracks. In our simulation, the size of the sliding window is 20, that may not be optimal for these two settings when determining to process a feature measurement as $\mathcal{C}_{f, \text{repj}}$ or $\mathcal{C}_{f, \text{pose}}$. However, this is solvable by finding the relationship between the camera motion and the feature track lengths as analyzed in [37], and thus should also be one of the directions in our future research.

VII. EXPERIMENTAL RESULTS

We further experimentally validate our proposed SLAM estimator and show its performance on real-world data. For our SLAM system, we develop a robocentric front-end based on our previous work, R-VIO2 [56], by enabling local mapping.⁹ For

⁹[Online]. Available: <https://github.com/rpng/R-VIO2>

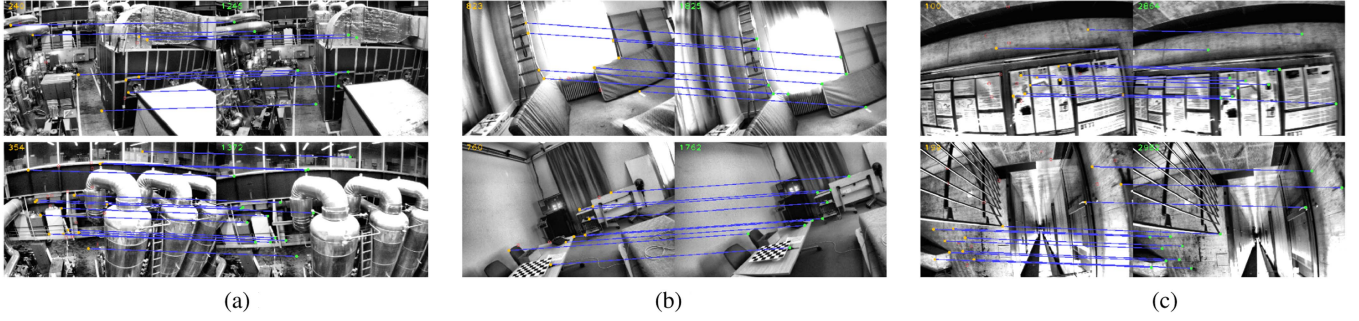


Fig. 4. Sampled loop-closure detection results (after outlier rejection) on the three datasets in our tests, where each blue line connects two matched features in two images (yellow: previous, green: current) captured at the same place. (a) EuRoC dataset. (b) Vicin Loops dataset. (c) CLA dataset.

the back-end, the in-place QR factorization used for real-time information update and marginalization is realized using Givens rotations in the Eigen library [65], and the nonreal-time batch optimization is implemented by Ceres where both sparse Cholesky factorization and Levenberg–Marquardt strategy are used to help solve the nonlinear BA problem. Loop-closure measurements are detected based on the place recognition by DBow2 [66] and feature matching by BRIEF descriptors [67] (see Fig. 4). A loop-closure detector is integrated into our back-end, not combining it with the feature tracker in the front-end, regarding that the computation of loop-closure detection, including DBow2 database query, BRIEF descriptor matching, and random sample consensus (RANSAC)-based outlier rejection, is time consuming. Notably, our proposed back-end is able to handle delayed loop-closure measurements as long as they are detected in the current sliding-window. Therefore, in our system loop-closure detection has sufficient time budget to be finished. The system algorithm is implemented with C++, where Robot Operating System¹⁰ is used to handle the communication between threads and visualize the output.

We use three representative datasets to evaluate the performance of our system. The first one is the *EuRoC* dataset which provides a benchmark of SLAM estimator in the scenarios of highly dynamic motions [68]. The next one is the *Vicin Loops* dataset that was collected in [15] to demonstrate the persistence of SLAM estimator in a long-term testing. The last one is the *CLA* dataset that was used in [61] for creating a prior map of a five-story building which is employed to verify the ability of SLAM estimator of closing loops in a large-scale environment. These datasets are complementary to each other which help to reveal issues and showcase the effectiveness of our proposed method. Especially, we compare our system against the state-of-the-art VINS-Mono.¹¹ All the tests run on a laptop with Intel Core i7-4710MQ 2.5 GHz×8 CPU and 16-GB RAM in *real time*.

A. *EuRoC* Dataset

The dataset consists of eleven sequences that are collected by a flying quadrotor equipped with a VI-sensor (one 200 Hz IMU

and two 20 Hz cameras of 752×480 resolution) [68]. The first six sequences are collected in a room filled with randomly placed objects where the trajectories of the quadrotor are captured by a Vicin system at 100 Hz, while the following five sequences are in a large machine hall where a Leica laser tracker is used to record the position of quadrotor. The groundtruth is generated offline using BA by incorporating those measures with sensor measurements. In this test, we only use the images from left camera as the visual input, and 300 Shi-Tomasi features [69] are extracted for each image. Especially, for VIO in our front-end the sliding window includes 15 most recent relative poses and the local map includes at most 50 feature points. Those feature tracks longer than the sliding window size will be cut into multiple sub-tracks for use. While for VINS-Mono, we keep its default settings to guarantee best performance on this dataset. Note that, we use the values provided by the dataset to initialize spatiotemporal sensor parameters for both systems.

We first study their performance on six-DOF pose estimation. It should be noted that we conducted similar studies in [16] and [56] which focused on the VIO parts of the two systems, while here we will assess both front-ends and back-ends. The six-DOF pose RMSEs against the groundtruth are presented with their respective medians based on the outputs of each estimator by running same sequence five times.¹² The estimated trajectories (and maps) are gravity-aligned, and thus are aligned with the groundtruth in terms of original position and yaw angle [70]. The comparison result is summarized as Table III. From that, we can clearly see that our front-end outperforms the front-end of VINS-Mono almost on all the sequences, while even the results of two exceptions are still very comparable. For back-ends, VINS-Mono performs better on most sequences. As the Euclidean coordinates (i.e., xyz) are used for expressing the feature points in our global map, it may lead to poor accuracy in position for feature points in the distance [57], and hence degrade the accuracy in pose estimation. Most of the features observed in the Machine Hall are located much farther than those in the small Vicin Room, therefore, the above explains the difference in the comparison between the two back-ends. It should also be noted that different with VINS-Mono which only estimates the poses of the selected keyframes in its back-end, our back-end estimates all the IMU poses and observed feature

¹⁰[Online]. Available: <https://www.ros.org/>

¹¹[Online]. Available: <https://github.com/HKUST-Aerial-Robotics/VINS-Mono>

¹²[Online]. Available: https://github.com/uzh-rpg/rpg_trajectory_evaluation

TABLE III
COMPARISON OF ABSOLUTE POSE ACCURACY (RMSE) ON EUROC DATASET

	Length	VINS-Mono (Front-end)		Ours (Front-end)		VINS-Mono (Back-end)		Ours (Back-end)	
		Orientation	Position	Orientation	Position	Orientation	Position	Orientation	Position
	[m]	[deg]	[m]	[deg]	[m]	[deg]	[m]	[deg]	[m]
V1_01_easy	58.6	5.56	0.079	5.79	0.065	5.27	0.049	5.67	0.038
V1_02_medium	75.9	2.29	0.086	2.35	0.084	2.06	0.048	2.01	0.056
V1_03_difficult	79.0	5.39	0.216	1.89	0.087	4.98	0.195	2.46	0.062
V2_01_easy	36.5	1.60	0.071	2.06	0.127	1.20	0.066	0.92	0.092
V2_02_medium	83.2	2.41	0.122	1.49	0.121	1.83	0.061	1.03	0.078
V2_03_difficult	86.1	3.50	0.249	1.15	0.184	3.15	0.213	0.92	0.143
MH_01_easy	80.6	0.80	0.162	1.55	0.160	1.09	0.056	1.49	0.121
MH_02_easy	73.5	1.15	0.213	1.09	0.157	0.86	0.058	0.57	0.105
MH_03_medium	130.9	1.20	0.191	1.38	0.177	0.74	0.061	1.15	0.072
MH_04_difficult	91.7	0.86	0.311	1.15	0.305	0.74	0.125	0.74	0.143
MH_05_difficult	97.6	1.38	0.263	1.15	0.329	1.03	0.126	0.46	0.239

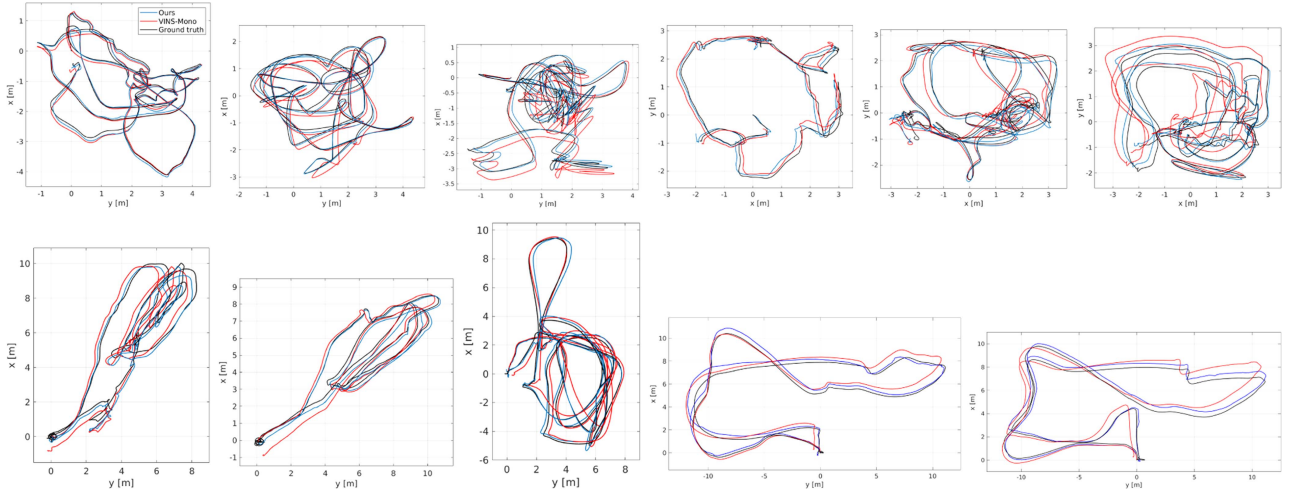


Fig. 5. Front-end estimated trajectories on EuRoC dataset: Ours (blue), VINS-Mono (red), and ground truth (black).

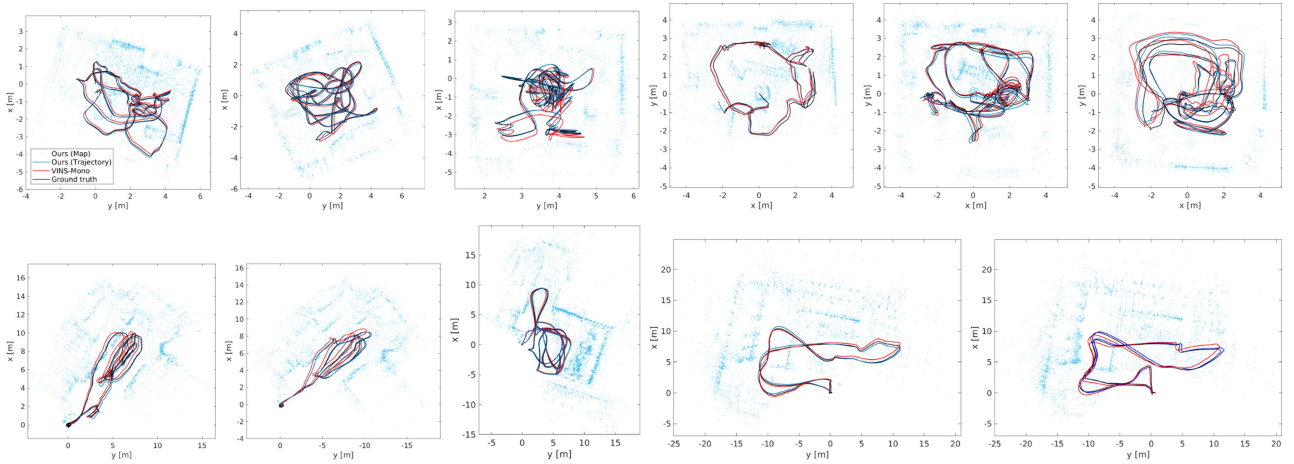


Fig. 6. Back-end estimated maps and trajectories on EuRoC dataset: Ours (blue), VINS-Mono (red), and ground truth (black).

points. Nevertheless, our back-end not only achieves improved accuracy as compared with the front-end, but also raises the front-end output to the same precision of accuracy by feeding back optimal loop-closure corrections. This shows that by taking into account estimation consistency in parallel estimator design our system now behaves similarly to a batch SLAM estimator

in terms of pose accuracy. We also depict the estimation results of the two systems and the groundtruths in Figs. 5 and 6 as the evidences to support our analysis.

We then investigate the time costs of the two systems. Our system runs on two threads when loop closure is not detected: One is for the VIO (in the front-end) and the other for building

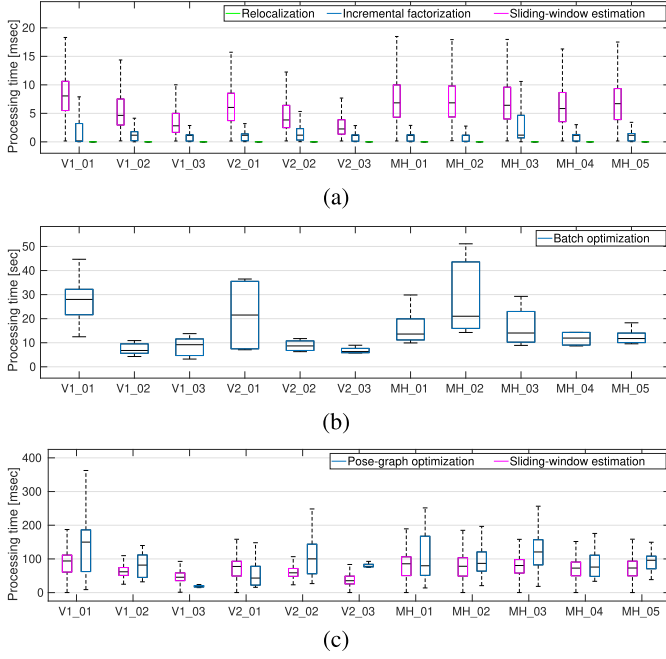


Fig. 7. Comparison of time consumption on EuRoC dataset. (a) Ours: Sliding-window estimation (purple, front-end), factorizing C_W and updating $\bar{C}_W$ (blue, back-end), and solving for $\delta\mathbf{x}_W^*$ and $\hat{\mathbf{x}}_W^*$ and performing relocalization (green, back-end). (b) Ours: Solving for $\delta\mathbf{x}_M^*$ and $\hat{\mathbf{x}}_M^*$ (blue, back-end). (c) VINS-Mono: Sliding-window estimation (purple, front-end) and pose-graph optimization (blue, back-end).

the graph of BA problem (in the back-end). Then, once finding the loop-closure measurements another thread will be created to solve the nonreal-time batch optimization. While for VINS-Mono, its front-end and back-end run on two separate threads, respectively. It should be noted that here we only focus on the estimator, therefore, the time costs on the feature tracking and loop-closure detection are excluded. However, for our system tracking 300 features takes 21 milliseconds in average, and the loop-closure measurements can be returned in 5 ms mostly depending on the size of DBoW database. The results are based on the statistics over 5 runs for each sequence,¹³ as shown in Fig. 7. It should be noted that for our system the time costs on the sliding-window estimation and the paralleled information factorization are less than 20 and 10 ms, respectively, and the composition-based relocalization step can be finished in negligible time. Those together guarantees real-time performance of our proposed parallel estimator. Note also that, our batch optimization for all the states in the map takes obviously longer time than the pose-graph optimization of VINS-Mono, which is as expected. Especially, the maximum number of iterations for Ceres solver is set to 50 in our setting to guarantee convergence, however, for VINS-Mono it is only 8 regardless of convergence. Those facts also contribute to the difference in pose accuracy between the back-ends as shown in Table III above, as our estimator performs far less number of loop closures than VINS-Mono. Nevertheless, as the real-time thread in our back-end is able

¹³ A standard box plot is used to describe the extremum, median, 1st (25%) and 3rd (75%) quartiles of a set of measures (e.g., the time costs over 5 runs).

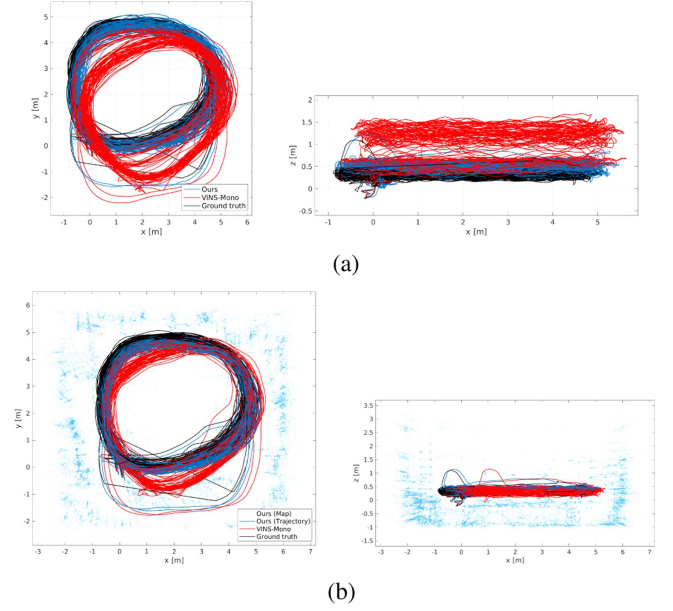


Fig. 8. Estimated map and trajectories on Vicon Loops dataset. (a) Estimated trajectories of the front-ends. (b) Estimated map (ours only) and trajectories of the back-ends.

to synchronize information with front-end during the global optimization, the loop closure that was initiated in a *previous* sliding window can still feed back consistent correction to the current sliding-window estimation.

B. Vicon-Loops Dataset

This dataset is collected in the same room as the previous dataset using the VI-sensor, however, IMU rate is increased to 800 Hz. The data collection lasts for almost 14 min by fast walking in circles at the speed around 2 m/s, and therefore it has many more loop closures than in the EuRoC dataset. This raises a great challenge to the SLAM estimator for generating consistent output. Also, the VI-sensor started in motion, which poses extra difficulties for system initialization. Beside those, the groundtruth is provided by a Vicon system at 200 Hz. Here, we use the same settings for EuRoC dataset for both systems, however, the number of features for each image is reduced to 200, regarding fast growth of the size of the graph.

The estimation results of the two systems are illustrated by Fig. 8. Especially, our estimated trajectory and map include 16408 IMU poses and 25107 feature points, respectively. We should note that our front-end and back-end results well fit the groundtruth in terms of both orientation and position, however, VINS-Mono suffers significant drifts in both orientation (e.g., yaw direction) and position (e.g., z -component). Our estimated map (without culling duplicate points) also renders the room with interior details, like ladder and tables. As both estimated trajectories are lengthy, here we use the same metrics as used in [15] and [26] for evaluating pose accuracy. Fig. 9 shows both pose error statistics and altitude changes. In particular, we group the pose errors based on the traveling distances that are increased from the origin and use box plots to depict the corresponding

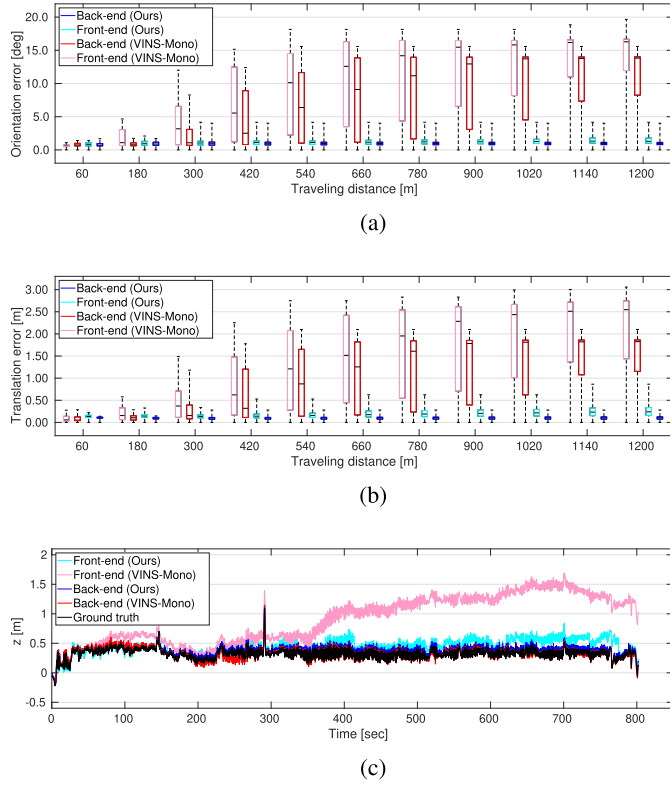


Fig. 9. Comparison of pose accuracy on Vicin Loops dataset. (a) Statistics (in the form of standard box plots) of the absolute orientation errors with respect to the traveling distances. (b) Statistics (in the form of standard box plots) of the absolute translation errors with respect to the traveling distances. (c) Altitude profile, that is the z -components of position outputs.

statistics, as shown in Fig. 9(a) and (b). It is easy to notice that for VINS-Mono the pose accuracy of front-end and back-end degrades following two very similar trends, meaning that its back-end (pose-graph) optimization failed to close the loops and did not provide effective corrections back to the front-end. In contrast to that, our system has the pose errors consistently at a low level, where the major orientation and translation errors are always below 2.5° and 0.5 m, respectively. The result of altitude in Fig. 9(c) further demonstrates superior performance of our system.

Furthermore, we investigate the processing capacity of the two systems. First, both front-ends perform similarly to what we shown on EuRoC dataset regarding similar settings used. Therefore, we are more interested in the performance of the two back-ends, especially on handling frequent loop closures. Fig. 10 depicts the growth of graph in terms of the numbers of nodes and edges, as SLAM goes on. For VINS-Mono, the number of nodes and edges increase almost *linearly* because only the keyframes' poses are included for batch optimization. Different with VINS-Mono, our system includes all the IMU poses and stably tracked features into batch optimization, and the size of graph grows almost *exponentially*. As a result, at any time of loop closure the size of the graph in our system is one order-of-magnitude greater than the size of the pose graph in VINS-Mono. The corresponding processing time of batch

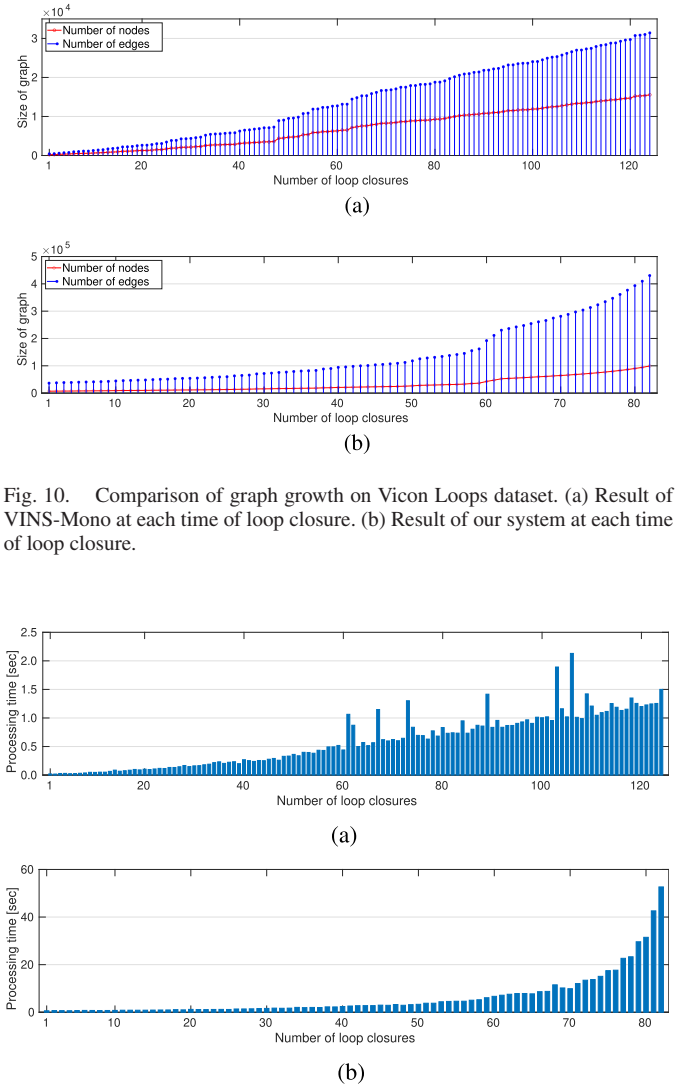


Fig. 10. Comparison of graph growth on Vicin Loops dataset. (a) Result of VINS-Mono at each time of loop closure. (b) Result of our system at each time of loop closure.

Fig. 11. Comparison of processing time of batch optimization on Vicin Loops dataset. (a) Result of VINS-Mono at each time of loop closure. (b) Result of our system at each time of loop closure.

optimization is also measured. Especially, here we force Ceres to perform only one iteration in solving the problem, mainly because the graph become dense only after a few loop closures which significantly slows down the convergence of the solver. As a result, the processing time of our batch optimization is nearly proportional to the size of the graph. Fig. 11 shows the processing time for the cases in Fig. 10, where, compared with VINS-Mono whose processing time increases linearly, ours increases exponentially as expected, following the growth of the graph. It should be noted that our batch optimization takes seconds to tens of seconds to be finished which is much longer than the time scope of one sliding window (e.g., our sliding window of size 15 corresponds to 0.75 s). This means our proposed parallel method can tolerate time delays of any length in processing loop closure, which is very useful to SLAM when dense mapping and frequent loop closure are performed at the same time, such as in AR/VR applications.

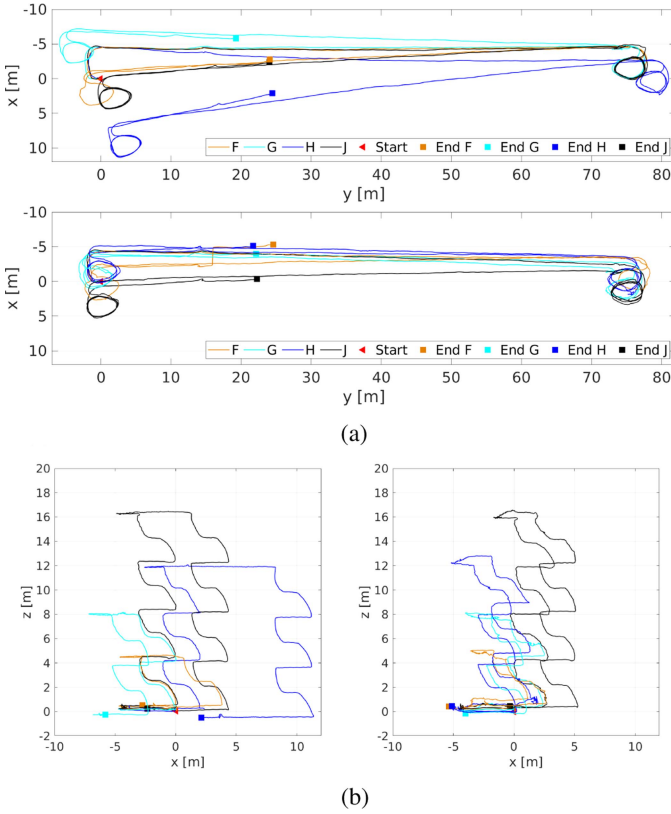


Fig. 12. Front-end estimated trajectories on CLA dataset. (a) Overhead plot: VINS-Mono (top) and our system (bottom). (b) Lateral plot: VINS-Mono (left) and our system (right).

C. CLA Dataset

This is a large indoor dataset including four closed-loop trails which span five floors. It is collected with a hand-held VI-sensor by walking in a building. IMU runs at 200 Hz and the cameras are both 20 Hz. The trajectory of each trail passes through the ground floor, staircases and one other floor (F, G, H or J) of the building, and together they cover over 1 km. In such case, loop closures occur always at the end of each trajectory when obvious pose drift may have shown up in the front-end estimate. Thus, our goal is to reveal the ability of an estimator on closing large loop and building consistent map. Especially, we resume the maximum number of iterations in Ceres solver to 50 for our batch optimization, and same for VINS-Mono.

The estimated trajectories of the two systems are shown in Figs. 12 and 13. Although there is no groundtruth provided, the interior of building can be considered to be aligned with a world frame. Therefore, we align all those trajectories to the same original pose that has zero rotation and zero translation. From Fig. 14(b) we should note that both front-ends suffer pose drift when traveling in the one-shot loops. Loop-closure corrections are applied to the front-ends, which can be found out in our results shown in 14(a) where the ends of trajectories F, G, and H jump back to their expected locations around the line $x = -5$. Trajectory J in our result does not close the loop which is mainly because it is the longest one among the four

and the batch optimization is not yet finished before reaching the end of the trajectory. In contrary, VINS-Mono cannot close any loops by its front end. The corresponding trajectories are biased with loop-closure corrections, which can be seen in the results of trajectory G and H. This shows that only a heuristic keyframe-based strategy can easily fail in this scenario where pose-only corrections ignored too much information about the correlation between front-end and back-end states. Notably the results of back-ends also support our analysis, where by taking into account all the IMU and camera measurements our results of trajectories overlap with each other in both horizontal and vertical directions. Similarly, the pose-graph optimization used in the back-end of VINS-Mono suffers obvious accuracy issue where most of the trajectories do not converge to the expected location ($x = -5$) and their scales drift when passing through the staircase on the right side. This is because by ignoring the features estimated in its front-end, the relative pose constraints in its back-end (pose graph) provide very limited information about the entire trajectory, especially for the loop closures. As a proof of performance, we also depicts our mapping result in Fig. 14 where we reconstruct 3460 poses and 7405 features in trajectory F, 4398 poses and 5718 features in trajectory G, 4908 poses and 6495 features in trajectory H, and 5593 poses and 8509 features in trajectory J, respectively.

VIII. CONCLUSION

In this article, we present a novel efficient SLAM framework which comprises of a generalized front-end used for real-time motion tracking and a back-end used for batch optimization of mapping with loop closures. Distinguished from the previous work on parallel SLAM estimators that focus on the functional decoupling for efficiency, we take into account the estimation consistency throughout our design, based on the batch SLAM estimation. We parallelize its functions into multiple threads, but always maintain the correlation between them. In this way, the front-end and back-end in our proposed parallel estimator are tightly coupled just like the batch estimator, which is well demonstrated in handling the loop closures. In particular, we have developed a hybrid parallel VI-SLAM estimator, where the robocentric and world-centric formulations are adopted by the respective estimations in the front-end and back-end, so as to reveal the versatility of our proposed framework. We have validated our method using both Monte-Carlo simulations and real experiments on three challenging datasets. The simulation results demonstrate that our proposed estimator is able to incorporate loop-closure measurements in a consistent manner such that the performance of VI-SLAM can be guaranteed to surpass a VIO. The experimental results further show that by following our design the front-end and back-end of the parallel SLAM estimator can achieve similar precision in accuracy in 3-D localization, and build consistent trajectories and maps by leveraging loop-closure measurements online.

The proposed VI-SLAM holds great potential for future research and applications. In particular, our method can tolerate significant time delay during the loop-closure processing, such

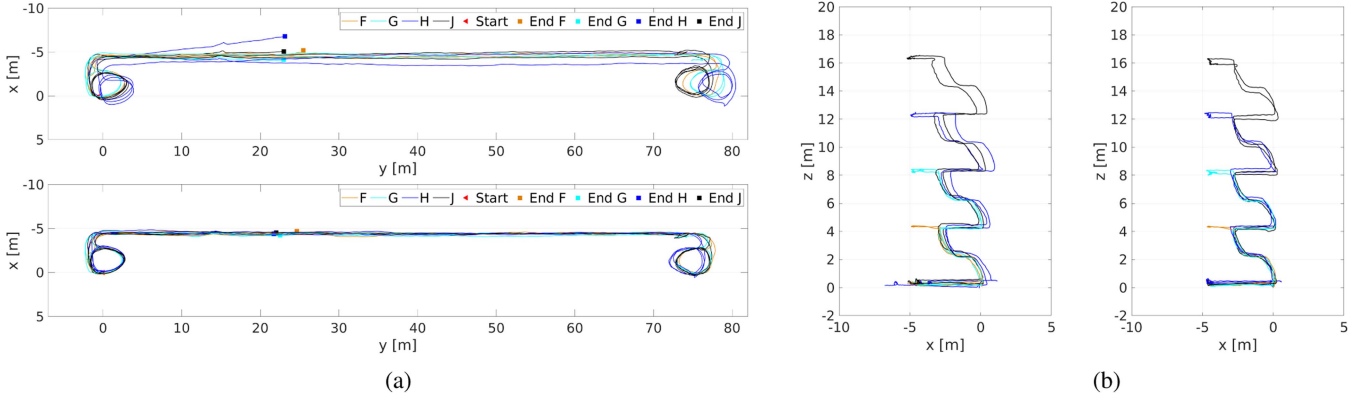


Fig. 13. Back-end estimated trajectories on CLA dataset. (a) Overhead plot: VINS-Mono (top) and our system (bottom). (b) Lateral plot: VINS-Mono (left) and our system (right).

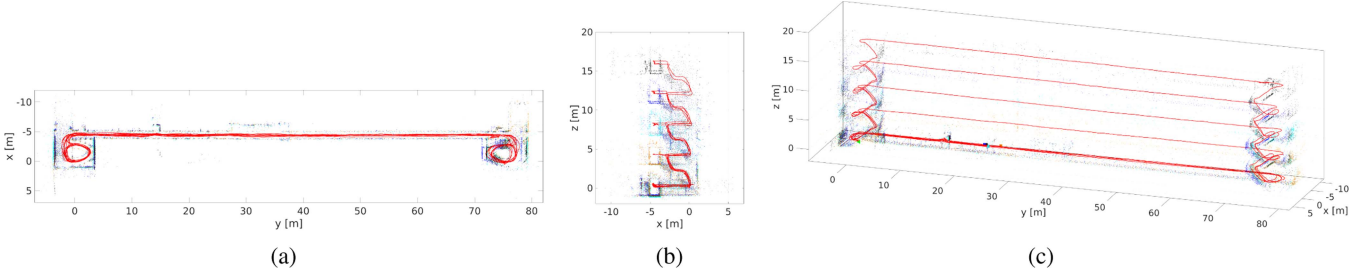


Fig. 14. Our estimated map on CLA dataset covering the corridors and staircases spanning five floors (ground, F, G, H, J). (a) Overhead view. (b) Lateral view. (c) Stereo view.

that some expensive operations, like marginalization and sparsification, are now able to be performed in online mapping, for example by running together with batch optimization, to boost the processing rate of back-end. Moreover, we can also extend our framework to implementing SLAM estimator as a distributed client-server system, where the front-ends run on users' devices like smartphones while the back-end processing all the client data runs on the cloud, to scale up applications.

APPENDIX A

INFORMATION FACTORS IN ROBOCENTRIC VIO

For the ensuing derivation, we employ an intermediate state (before composition) with respect to $\{R_k\}$ at timestep $k + 1$

$$\hat{\mathbf{x}}_{k+1} = \begin{bmatrix} \hat{\mathbf{x}}_{\mathcal{F}_k}^\top & \hat{\mathbf{x}}_{\mathcal{G}_k}^\top & \hat{\mathbf{x}}_{\mathcal{P}_k}^\top & \hat{\mathbf{x}}_{\mathcal{W}_{k+1}}^\top \end{bmatrix}^\top \quad (52)$$

where the relative motion estimate between $\{R_k\}$ and $\{I_{k+1}\}$, $\hat{\mathbf{x}}_{w_N}$, which is computed using the IMU measurements in the time interval $[t_k + t_d, t_{k+1} + t_d]$ has been appended to $\hat{\mathbf{x}}_{\mathcal{W}_k}$ to form the current sliding window $\hat{\mathbf{x}}_{\mathcal{W}_{k+1}}$ [56].

A. IMU Factor

$\mathcal{C}_{\text{IMU}}$ remains the same as derived in [56].

B. Camera Factors

Specifically, we assume a feature f can be observed from a series of images $\mathcal{S}$ (e.g., $\{k - N, \dots, k + 1\}$). Then using the perspective projection model, the i th ($i \in \mathcal{S}$) measurement of f can be expressed as

$$\mathbf{z}_i = \frac{1}{z_i(t + t_d)} \begin{bmatrix} x_i(t + t_d) \\ y_i(t + t_d) \end{bmatrix} + \mathbf{n}_{\text{im}} \quad (53)$$

$${}^{C_i}\mathbf{p}_f = \begin{bmatrix} x_i(t + t_d) & y_i(t + t_d) & z_i(t + t_d) \end{bmatrix}^\top \quad (54)$$

where $\mathbf{n}_{\text{im}} \sim \mathcal{N}(\mathbf{0}_{2 \times 1}, \sigma_{\text{im}}^2 \mathbf{I}_2)$ is the image noise, and ${}^{C_i}\mathbf{p}_f$ is the position of the feature in the corresponding camera frame $\{C_i\}$, while here we explicitly align it with the actual pose of camera at image timestamp t by taking into account t_d .

a) *Local feature initialization*: We use the same inverse-depth parameterization as used by R-VIO2 to initialize newly observed features. Especially, following the derivation in [56] we can get the cost function of all M ($M = |\mathcal{S}|$) measurements of the feature f with respect to its inverse-depth form λ ¹⁴

$$\mathcal{C}_f = \|\mathbf{H}_\chi \delta \chi + \mathbf{H}_\lambda \delta \lambda - \mathbf{e}_f\|_{\Sigma_f}^2 \quad (55)$$

where $\mathbf{H}_\chi$, $\mathbf{H}_\lambda$ and $\mathbf{e}_f$ are the stacked Jacobians and residual, and $\Sigma_f = \sigma_{\text{im}}^2 \mathbf{I}_{2M}$ is the stacked covariance matrix. By noting

¹⁴We have $\lambda := [\phi, \psi, \rho]^\top$, where ϕ and ψ are the elevation and azimuth angles, respectively, and ρ is the inverse of the depth of f in $\{C\}$ [56].

that $\mathbf{H}_\lambda$ is a tall matrix for $M \geq 2$, its QR factorization result is used to separate the information in $\mathcal{C}_f$

$$\begin{aligned} \mathcal{C}_f &= \left\| \mathbf{H}_\lambda \delta \chi + \begin{bmatrix} \mathbf{Q}_1 & \mathbf{Q}_2 \end{bmatrix} \begin{bmatrix} \mathbf{R}_\lambda \\ \mathbf{0} \end{bmatrix} \delta \lambda - \mathbf{e}_f \right\|_{\Sigma_f}^2 \\ &= \left\| \begin{bmatrix} \mathbf{Q}_1^\top \\ \mathbf{Q}_2^\top \end{bmatrix} \mathbf{H}_\lambda \delta \chi + \begin{bmatrix} \mathbf{R}_\lambda \\ \mathbf{0} \end{bmatrix} \delta \lambda - \begin{bmatrix} \mathbf{Q}_1^\top \\ \mathbf{Q}_2^\top \end{bmatrix} \mathbf{e}_f \right\|_{\Sigma_f}^2 \\ &= \|\mathbf{Q}_1^\top \mathbf{H}_\lambda \delta \chi + \mathbf{R}_\lambda \delta \lambda - \mathbf{Q}_1^\top \mathbf{e}_f\|_{\Sigma_{f,1}}^2 \\ &\quad + \|\mathbf{Q}_2^\top \mathbf{H}_\lambda \delta \chi - \mathbf{Q}_2^\top \mathbf{e}_f\|_{\Sigma_{f,2}}^2 \end{aligned} \quad (56)$$

where $\Sigma_{f,1} = \sigma_{\text{im}}^2 \mathbf{I}_3$ and $\Sigma_{f,2} = \sigma_{\text{im}}^2 \mathbf{I}_{2M-3}$. Especially, the resultant augend encodes the information of the feature f and its correlation with the current state χ , while the addend only reflects the relationship between different poses in the current sliding window. As a result, we obtain two square-root factors

$$\mathcal{C}_{f,\text{init}} = \|\mathbf{R}_{\lambda\lambda} \delta \lambda + \mathbf{R}_{\lambda\chi} \delta \chi - \mathbf{r}_\lambda\|^2 \quad (57)$$

$$\mathcal{C}_{f,\text{pose}} = \|\mathbf{J}_{f,p} \delta \chi - \mathbf{r}_{f,p}\|^2 \quad (58)$$

where $\mathbf{R}_{\lambda\lambda} = \sigma_{\text{im}}^{-1} \mathbf{R}_\lambda$, $\mathbf{R}_{\lambda\chi} = \sigma_{\text{im}}^{-1} \mathbf{Q}_1^\top \mathbf{H}_\lambda$, $\mathbf{r}_\lambda = \sigma_{\text{im}}^{-1} \mathbf{Q}_1^\top \mathbf{e}_f$, and $\mathbf{J}_{f,p} = \sigma_{\text{im}}^{-1} \mathbf{Q}_2^\top \mathbf{H}_\lambda$, $\mathbf{r}_{f,p} = \sigma_{\text{im}}^{-1} \mathbf{Q}_2^\top \mathbf{e}_f$. Note that $\mathbf{R}_{\lambda\lambda}$ is upper-triangular. We initialize a feature when its track spans an entire sliding window (i.e., $|\mathcal{S}| = N + 1$) for the first time.

b) Multiview pose constraint: For the features of $M \geq 2$, we stack their $\mathcal{C}_{f,\text{pose}}$ as in [56] for the VIO estimation.

c) Global reprojection constraint: If f has been included in the state $\mathbf{x}_f$ [see (19)], we can express its local position in the current camera frame $\{C_{k+1}\}$

$$\begin{aligned} {}^{C_{k+1}}\mathbf{p}_f &= {}^C \mathbf{C}_{\hat{q}}^{I_{k+1}} \mathbf{p}_f + {}^C \mathbf{p}_I \\ &= {}^C \mathbf{C}_{\hat{q}_k}^{k+1} \mathbf{C}_{\hat{q}}^k ({}_G^k \mathbf{C}_{\hat{q}}^G \mathbf{p}_f + {}^{R_k} \mathbf{p}_G - {}^{R_k} \mathbf{p}_{I_{k+1}}) + {}^C \mathbf{p}_I \end{aligned} \quad (59)$$

which is a function of the current state χ_{k+1} . By linearizing (53) at $\hat{\chi}_{k+1}$ except that for ${}^G \mathbf{p}_f$ we use its FEJ estimate, the reprojection error of f is given by

$$\mathbf{e}_{k+1} = \mathbf{z}_{k+1} - \hat{\mathbf{z}}_{k+1} \simeq \mathbf{H}_{\chi_{k+1}} \delta \chi_{k+1} + \mathbf{n}_{\text{im}} \quad (60)$$

where $\mathbf{H}_{\chi_{k+1}}$ is derived in Appendix B. Accordingly, we have the reprojection-error factor of f :

$$\mathcal{C}_{f,\text{repj}} = \|\mathbf{H}_{\chi_{k+1}} \delta \chi_{k+1} - \mathbf{e}_{k+1}\|_{\Sigma_f}^2 = \|\mathbf{J}_{f,r} \delta \chi_{k+1} - \mathbf{r}_{f,r}\|_{\Sigma_f}^2 \quad (61)$$

where $\Sigma_f = \sigma_{\text{im}}^2 \mathbf{I}_2$, $\mathbf{J}_{f,r} = \sigma_{\text{im}}^{-1} \mathbf{H}_{\chi_{k+1}}$ and $\mathbf{r}_{f,r} = \sigma_{\text{im}}^{-1} \mathbf{e}_{k+1}$.

APPENDIX B

EXPRESSIONS OF JACOBIANS IN (60)

The components of Jacobian $\mathbf{H}_{\chi_{k+1}}$ are shown as follows:

$$\mathbf{H}_{\chi_{k+1}} = \mathbf{H}_{\text{proj}} \begin{bmatrix} \mathbf{H}_f & \mathbf{H}_g & \mathbf{H}_p & \mathbf{H}_w \end{bmatrix} \quad (62)$$

$$\mathbf{H}_{\text{proj}} = \hat{\mathbf{z}}_{k+1}^{-1} \begin{bmatrix} 1 & 0 & -\hat{x}_{k+1} \hat{z}_{k+1}^{-1} \\ 0 & 1 & -\hat{y}_{k+1} \hat{z}_{k+1}^{-1} \end{bmatrix} \quad (63)$$

1) *Jacobian W.r.t. $\mathbf{x}_f$:* Only the block corresponding to ${}^G \mathbf{p}_f$ is nonzero in $\mathbf{H}_f = [\mathbf{0}_{2 \times 3} \cdots \mathbf{H}_f \mathbf{0}_{2 \times 3} \cdots]$

$$\mathbf{H}_f = \frac{\partial {}^{C_{k+1}} \mathbf{p}_f}{\partial {}^G \mathbf{p}_f} = {}^C \mathbf{C}_{\hat{q}_k}^{k+1} \mathbf{C}_{\hat{q}_G}^k \mathbf{C}_{\hat{q}}^G \quad (64)$$

2) *Jacobian W.r.t. $\mathbf{x}_g$:* $\mathbf{H}_g = [\mathbf{H}_{\theta_G} \mathbf{H}_{p_G} \mathbf{0}_{2 \times 3}]$, where

$$\mathbf{H}_{\theta_G} = \frac{\partial {}^{C_{k+1}} \mathbf{p}_f}{\partial \delta_k^G \boldsymbol{\theta}} = {}^C \mathbf{C}_{\hat{q}_k}^{k+1} \mathbf{C}_{\hat{q}}^k [{}_G^k \mathbf{C}_{\hat{q}}^G \hat{\mathbf{p}}_f] \quad (65)$$

$$\mathbf{H}_{p_G} = \frac{\partial {}^{C_{k+1}} \mathbf{p}_f}{\partial {}^{R_k} \mathbf{p}_G} = {}^C \mathbf{C}_{\hat{q}_k}^{k+1} \mathbf{C}_{\hat{q}}^k \quad (66)$$

3) *Jacobian W.r.t. $\mathbf{x}_w$:* We compute this Jacobian first, as it will be used later. Especially, only the block corresponding to $\mathbf{x}_{w_N}$ is nonzero in $\mathbf{H}_w = [\mathbf{0}_{2 \times 6} \cdots \mathbf{0}_{2 \times 15} \mathbf{H}_{w_N}]$

$$\mathbf{H}_{w_N} = [\mathbf{H}_{\theta_{k+1}} \mathbf{H}_{p_{k+1}} \mathbf{0}_{2 \times 9}] \quad (67)$$

$$\mathbf{H}_{\theta_{k+1}} = \frac{\partial {}^{C_{k+1}} \mathbf{p}_f}{\partial \delta_k^{k+1} \boldsymbol{\theta}} = {}^C \mathbf{C}_{\hat{q}}^{k+1} [{}^{I_{k+1}} \hat{\mathbf{p}}_f] \quad (68)$$

$$\mathbf{H}_{p_{k+1}} = \frac{\partial {}^{C_{k+1}} \mathbf{p}_f}{\partial {}^{R_k} \mathbf{p}_{I_{k+1}}} = -{}^C \mathbf{C}_{\hat{q}_k}^{k+1} \mathbf{C}_{\hat{q}}^k \quad (69)$$

4) *Jacobian W.r.t. $\mathbf{x}_p$:* $\mathbf{H}_p = [\mathbf{H}_{\theta_C} \mathbf{H}_{p_C} \mathbf{H}_{t_d}]$, where

$$\mathbf{H}_{\theta_C} = \frac{\partial {}^{C_{k+1}} \mathbf{p}_f}{\partial \delta_I^C \boldsymbol{\theta}} = [{}_I^C \mathbf{C}_{\hat{q}}^{I_{k+1}} \hat{\mathbf{p}}_f] \quad (70)$$

$$\mathbf{H}_{p_C} = \frac{\partial {}^{C_{k+1}} \mathbf{p}_f}{\partial {}^C \mathbf{p}_I} = \mathbf{I}_3 \quad (71)$$

$$\begin{aligned} \mathbf{H}_{t_d} &= \frac{\partial {}^{C_{k+1}} \mathbf{p}_f}{\partial \delta_k^{k+1} \boldsymbol{\theta}} \frac{\partial \delta_k^{k+1} \boldsymbol{\theta}}{\partial t_d} + \frac{\partial {}^{C_{k+1}} \mathbf{p}_f}{\partial {}^{R_k} \mathbf{p}_{I_{k+1}}} \frac{\partial {}^{R_k} \mathbf{p}_{I_{k+1}}}{\partial t_d} \\ &= {}^C \mathbf{C}_{\hat{q}}^{k+1} [{}^{I_{k+1}} \hat{\mathbf{p}}_f] {}^{I_{k+1}} \hat{\boldsymbol{\omega}} - {}^C \mathbf{C}_{\hat{q}}^{I_{k+1}} \hat{\mathbf{v}}. \end{aligned} \quad (72)$$

APPENDIX C

ESTIMATION IN ROBOCENTRIC VIO

We perform the sliding-window MAP estimation similar to R-VIO2 [56]. The prior factor containing the information up to the current timestep $k + 1$ [see (19) and (52)] is given by

$$\mathcal{C}_{\text{Prior},k+1} = \|\mathbf{R}_k \delta \mathbf{x}_k + \mathbf{d}_k\|^2 = \|\mathbf{R}_{k+1} \delta \chi_{k+1} + \mathbf{d}_k\|^2 \quad (73)$$

where $\mathbf{R}_{k+1} = [\mathbf{R}_k \mathbf{0}_{\gamma \times 15}]$ with γ the dimension of $\mathbf{R}_k$. We first fuse the information from IMU and new local features, and then global features, in order to keep observability properties of robocentric VIO for estimation consistency. After update, we perform composition to generate new features and update global pose estimate, followed by the marginalization.

A. Information Update

In this step, the IMU factor and multiview pose constraint factors from all the qualified features are used for information update. The VIO cost function for timestep $k + 1$ is formed as

follows:

$$\begin{aligned} \mathcal{C}_{VIO,k+1} &= \mathcal{C}_{\text{Prior},k+1} + \mathcal{C}_{I,k+1} + \sum_{i=1}^m \mathcal{C}_{f_i,\text{pose}} \\ &= \left\| \begin{bmatrix} \mathbf{R}_{k+1} \\ \mathbf{J}_I \\ \mathbf{J}_{f_1,p} \\ \vdots \\ \mathbf{J}_{f_m,p} \end{bmatrix} \delta \mathbf{x}_{k+1} - \begin{bmatrix} \mathbf{d}_k \\ \mathbf{r}_I \\ \mathbf{r}_{f_1,p} \\ \vdots \\ \mathbf{r}_{f_m,p} \end{bmatrix} \right\|^2 \end{aligned} \quad (74)$$

where we assume there exist m qualified features. We perform in-place QR factorization to refactor (74) as in (9)

$$\mathcal{C}_{VIO,k+1} = \|\mathbf{R}_{k+1}^{\oplus} \delta \mathbf{x}_{k+1} - \mathbf{d}_{k+1}^{\oplus}\|^2 \quad (75)$$

where the dimension of $\mathbf{R}_{k+1}^{\oplus}$ is $\gamma + 15$. Next, we incorporate the information from the new initialized features. Assuming s ($s \leq m$) new features are initialized at timestep $k + 1$, we add them into VIO cost function in the following way:

$$\begin{aligned} \mathcal{C}_{VIO,k+1}^+ &= \mathcal{C}_{VIO,k+1} + \sum_{i=1}^s \mathcal{C}_{f_i,\text{init}} \\ &= \left\| \begin{bmatrix} \mathbf{R}_{\lambda_1 \lambda_1} & \cdots & \mathbf{0}_3 & \mathbf{R}_{\lambda_1 \chi} \\ & \ddots & \vdots & \vdots \\ & & \mathbf{R}_{\lambda_s \lambda_s} & \mathbf{R}_{\lambda_s \chi} \\ & & & \mathbf{R}_{k+1}^{\oplus} \end{bmatrix} \begin{bmatrix} \delta \lambda_1 \\ \vdots \\ \delta \lambda_s \\ \delta \mathbf{x}_{k+1} \end{bmatrix} - \begin{bmatrix} \mathbf{r}_{\lambda_1} \\ \vdots \\ \mathbf{r}_{\lambda_s} \\ \mathbf{d}_{k+1}^{\oplus} \end{bmatrix} \right\|^2 \\ &= \|\mathbf{R}_{k+1}^+ \delta \mathbf{x}_{k+1}^+ - \mathbf{d}_{k+1}^+\|^2 \end{aligned} \quad (76)$$

where $\mathbf{R}_{k+1}^{\oplus}$ and $\mathbf{d}_{k+1}^{\oplus}$ are augmented to $\mathbf{R}_{k+1}^+$ and $\mathbf{d}_{k+1}^+$, and we obtain the augmented state vector: $\hat{\mathbf{x}}_{k+1}^+ = [\hat{\boldsymbol{\lambda}}_{k+1}^{\top} \hat{\mathbf{x}}_{k+1}^{\top}]^{\top}$ with $\boldsymbol{\lambda}_{k+1}^{\top} = [\lambda_1^{\top} \cdots \lambda_s^{\top}]^{\top}$, whose error is $\delta \mathbf{x}_{k+1}^+$ accordingly. Especially, we leave the state update to the next step as more information from the global features will be included.

B. State-Only Update

As the use of global features should alter the observability properties of R-VIO2 and lead to inconsistency, we will only use global feature measurements to update the state estimates in the front-end, inspired by [71]. To this end, we first add the square-root factors of the global features to VIO cost function

$$\begin{aligned} \mathcal{C}_{VIO,k+1}^* &= \mathcal{C}_{VIO,k+1}^+ + \sum_{i=m+1}^{m+n} \mathcal{C}_{f_i,\text{repj}} \\ &= \left\| \begin{bmatrix} \mathbf{R}_{k+1}^+ \\ \mathbf{0}_{2 \times 3s} & \mathbf{J}_{f_{m+1},r} \\ \vdots & \vdots \\ \mathbf{0}_{2 \times 3s} & \mathbf{J}_{f_{m+n},r} \end{bmatrix} \delta \mathbf{x}_{k+1}^+ - \begin{bmatrix} \mathbf{d}_{k+1}^+ \\ \mathbf{r}_{f_{m+1},r} \\ \vdots \\ \mathbf{r}_{f_{m+n},r} \end{bmatrix} \right\|^2 \end{aligned} \quad (77)$$

where we assume there are n global feature measurements. In particular, here we perform in-place QR factorization as in the

preceding step to update cost function

$$\mathcal{C}_{VIO,k+1}^* = \|\mathbf{R}_{k+1}^* \delta \mathbf{x}_{k+1}^+ - \mathbf{d}_{k+1}^*\|^2 \quad (78)$$

where the dimension of $\mathbf{R}_{k+1}^*$ is same as $\mathbf{R}_{k+1}^+$. We minimize this cost function to solve for the optimal state update, as

$$\delta \hat{\mathbf{x}}_{k+1}^{+*} = \arg \min_{\delta \mathbf{x}_{k+1}^+} \mathcal{C}_{VIO,k+1}^* = \mathbf{R}_{k+1}^{*-1} \mathbf{d}_{k+1}^* \quad (79)$$

and thus, the optimal state estimate: $\hat{\mathbf{x}}_{k+1}^{+*} = \hat{\mathbf{x}}_{k+1}^+ \boxplus \delta \hat{\mathbf{x}}_{k+1}^{+*}$.

After such state update, we need to recover the information factor of VIO, $\mathcal{C}_{VIO,k+1}^+$, but now with respect to the updated state estimate, $\hat{\mathbf{x}}_{k+1}^{+*}$. Specifically, for such linearization point change we have the following derivation:

$$\begin{aligned} \mathcal{C}_{VIO,k+1}^+ &= \|\mathbf{R}_{k+1}^+ \delta \mathbf{x}_{k+1}^+ - \mathbf{d}_{k+1}^+\|^2 \\ &= \|\mathbf{R}_{k+1}^+ \delta \mathbf{x}_{k+1}^{+*} - \mathbf{d}_{k+1}^+\|^2 \\ &= \|\mathbf{R}_{k+1}^+ (\hat{\mathbf{x}}_{k+1}^+ + \delta \mathbf{x}_{k+1}^+ - \hat{\mathbf{x}}_{k+1}^{+*}) - \mathbf{d}_{k+1}^+\|^2 \\ &= \|\mathbf{R}_{k+1}^+ \delta \mathbf{x}_{k+1}^+ - \mathbf{R}_{k+1}^+ \delta \hat{\mathbf{x}}_{k+1}^{+*} - \mathbf{d}_{k+1}^+\|^2. \end{aligned} \quad (80)$$

Therefore, $\mathbf{d}_{k+1} = \mathbf{d}_{k+1}^+ - \mathbf{R}_{k+1}^+ \delta \hat{\mathbf{x}}_{k+1}^{+*}$, and VIO cost function with respect to the current state estimate, $\hat{\mathbf{x}}_{k+1}^{+*}$, is

$$\mathcal{C}_{VIO,k+1}^+ = \|\mathbf{R}_{k+1}^+ \delta \mathbf{x}_{k+1}^+ - \mathbf{d}_{k+1}\|^2. \quad (81)$$

C. Composition

As a distinguishing feature from the literature, composition is used by our robocentric VIO to update global state estimate and generate new global features. First, we perform composition for the global state $\hat{\mathbf{x}}_{\mathcal{G}_k}^* \xrightarrow{\hat{\mathbf{x}}_{wN}^*} \hat{\mathbf{x}}_{\mathcal{G}_{k+1}}$

$${}_{G}^{k+1} \hat{\mathbf{q}} = {}_k^{k+1} \hat{\mathbf{q}}^* \otimes {}_G^k \hat{\mathbf{q}}^* \quad (82)$$

$${}_{R_{k+1}} \hat{\mathbf{p}}_G = {}_k^{k+1} \mathbf{C}_{\hat{\mathbf{q}}}^* ({}_{R_k} \hat{\mathbf{p}}_G^* - {}_{R_k} \hat{\mathbf{p}}_{R_{k+1}}^*) \quad (83)$$

$${}_{R_{k+1}} \hat{\mathbf{g}} = {}_k^{k+1} \mathbf{C}_{\hat{\mathbf{q}}}^* {}_{R_k} \hat{\mathbf{g}}^*. \quad (84)$$

Note that, the change of robocentric frame of reference from $\{R_k\}$ to $\{R_{k+1}\}$ does not affect the other states such that we have $\hat{\mathbf{x}}_{\mathcal{P}_{k+1}} = \hat{\mathbf{x}}_{\mathcal{P}_k}^*$ and $\hat{\mathbf{x}}_{\mathcal{W}_{k+1}} = \hat{\mathbf{x}}_{\mathcal{W}_{k+1}}^*$. For $\hat{\boldsymbol{\lambda}}_{k+1}^*$, the s new features anchored on $\{C_{k+1}\}$, we transform them to the global frame $\{G\}$ so that there is no need to shift their anchor frame when $\{C_{k+1}\}$ moves out of the sliding window. Second, we show global feature generation $\hat{\boldsymbol{\lambda}}_{k+1}^* \xrightarrow{\hat{\mathbf{x}}_{\mathcal{G}_{k+1}}, \hat{\mathbf{x}}_{\mathcal{P}_{k+1}}} \hat{\mathbf{x}}_{F_{k+1}}^*$

$$\begin{aligned} {}_G \hat{\mathbf{p}}_{f_i}^* &= {}_{k+1}^G \mathbf{C}_{\hat{\mathbf{q}}}^* {}_{R_{k+1}} \hat{\mathbf{p}}_{f_i}^* + {}_G \hat{\mathbf{p}}_{R_{k+1}} \\ &= {}_{k+1}^G \mathbf{C}_{\hat{\mathbf{q}}}^* \left({}_I^G \mathbf{C}_{\hat{\mathbf{q}}}^* \frac{1}{\hat{\rho}_i^*} \mathbf{u}(\hat{\phi}_i^*, \hat{\psi}_i^*) + {}^I \hat{\mathbf{p}}_C \right) + {}_G \hat{\mathbf{p}}_{R_{k+1}} \\ &= {}_{k+1}^G \mathbf{C}_{\hat{\mathbf{q}}}^* \left[{}_I^G \mathbf{C}_{\hat{\mathbf{q}}}^* \left(\frac{1}{\hat{\rho}_i^*} \mathbf{u}(\hat{\phi}_i^*, \hat{\psi}_i^*) - {}^C \hat{\mathbf{p}}_I \right) - {}_{R_{k+1}} \hat{\mathbf{p}}_G \right] \end{aligned} \quad (85)$$

where $\hat{\boldsymbol{\lambda}}_i^* = [\hat{\phi}_i^*, \hat{\psi}_i^*, \hat{\rho}_i^*]^{\top}$, for $i = 1, \dots, s$. Accordingly, we have $\hat{\mathbf{x}}_{\mathcal{F}_{k+1}} = [\hat{\mathbf{x}}_{F_{k+1}}^{\top} \hat{\mathbf{x}}_{\mathcal{F}_k}^{\top}]^{\top}$. At this point, the composition for the state at timestep $k + 1$, $\mathbf{x}_{k+1}^+ \rightarrow \mathbf{x}_{k+1}$, is finished.

On the other hand, the square root information matrix of the VIO should also be updated along with the state composition. Specifically, here we follow [56, Lemma 1] to compute the Jacobians of $\mathbf{x}_{k+1}$ with respect to χ_{k+1}^+ . We should note that the composition for the new features uses the updated global state estimate. Thus, we should perform the information matrix composition in the same order. Especially, given the respective Jacobian matrices for the global state composition and global feature generation, $\mathbf{U}_G$ and $\mathbf{U}_F$, the composition on the square root information matrix $\mathbf{R}_{k+1}^+$ can be achieved by

$$\mathbf{R}_{k+1} = \mathbf{R}_{k+1}^+ \mathbf{U}_G^{-1} \mathbf{U}_F^{-1} \quad (86)$$

for which the detailed derivation is given in Appendix D. As a result, VIO cost function after composition becomes

$$\mathcal{C}_{\text{VIO},k+1} = \|\mathbf{R}_{k+1} \delta \mathbf{x}_{k+1} - \mathbf{d}_{k+1}\|^2. \quad (87)$$

D. Marginalization

We further follow Lemma 2 in [56] to reshape $\mathcal{C}_{\text{VIO},k+1}$ and $\mathbf{x}_{k+1}$ for marginalization. Once finishing that, the robocentric VIO estimation at timestep $k+1$ is completed.

APPENDIX D EXPRESSIONS OF JACOBIANS IN (86)

The Jacobian matrix of $\mathbf{x}_{k+1}$ with respect to χ_{k+1}^+ can be computed as two concatenate parts. First, the Jacobian matrix for the global state composition

$$\mathbf{U}_G = \begin{bmatrix} \mathbf{I}_{3s} & \mathbf{0}_{3s \times 3L} & \mathbf{0}_{3s \times 9} & \mathbf{0}_{3s \times 7} & \mathbf{0}_{3s \times 6N+18} \\ & \mathbf{I}_{3L} & \mathbf{0}_{3L \times 9} & \mathbf{0}_{3L \times 7} & \mathbf{0}_{3L \times 6N+18} \\ & & \mathbf{V}_{GG} & \mathbf{0}_{9 \times 7} & \mathbf{V}_{GW} \\ & & & \mathbf{I}_7 & \mathbf{0}_{7 \times 6N+18} \\ & & & & \mathbf{I}_{6N+18} \end{bmatrix}. \quad (88)$$

According to (82), (83), and (84), we have Jacobian matrices

$$\mathbf{V}_{GG} = \frac{\partial \mathbf{x}_{G,k+1}}{\partial \mathbf{x}_{G,k}} = \begin{bmatrix} {}^{k+1}_k \mathbf{C}_{\hat{q}} & \mathbf{0}_3 & \mathbf{0}_3 \\ & {}^{k+1}_k \mathbf{C}_{\hat{q}} & \mathbf{0}_3 \\ & & {}^{k+1}_k \mathbf{C}_{\hat{q}} \end{bmatrix} \quad (89)$$

$$\mathbf{V}_{GW} = \frac{\partial \mathbf{x}_{G,k+1}}{\partial \mathbf{x}_{W,k+1}} = [\mathbf{0}_{9 \times 6(N-1)+9} \quad \mathbf{V}_{wN}], \text{ where}$$

$$\mathbf{V}_{wN} = \frac{\partial \mathbf{x}_{G,k+1}}{\partial \mathbf{x}_{wN}} = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_3 & \mathbf{0}_{3 \times 9} \\ \mathbf{R}_{k+1} \hat{\mathbf{p}}_G & -{}^{k+1}_k \mathbf{C}_{\hat{q}} & \mathbf{0}_{3 \times 9} \\ \mathbf{R}_{k+1} \hat{\mathbf{g}} & \mathbf{0}_3 & \mathbf{0}_{3 \times 9} \end{bmatrix}. \quad (90)$$

Second, the Jacobian matrix for the global feature generation

$$\mathbf{U}_F = \begin{bmatrix} \mathbf{V}_{F\lambda} & \mathbf{0}_{3s \times 3L} & \mathbf{V}_{FG} & \mathbf{V}_{FP} & \mathbf{0}_{3s \times 6N+18} \\ & \mathbf{I}_{3L} & \mathbf{0}_{3L \times 9} & \mathbf{0}_{3L \times 7} & \mathbf{0}_{3L \times 6N+18} \\ & & \mathbf{I}_9 & \mathbf{0}_{9 \times 7} & \mathbf{0}_{9 \times 6N+18} \\ & & & \mathbf{I}_7 & \mathbf{0}_{7 \times 6N+18} \\ & & & & \mathbf{I}_{6N+18} \end{bmatrix}. \quad (91)$$

Using (85), the Jacobian matrices of ${}^G \mathbf{p}_{f_i}$ with respect to λ_i , $\mathbf{x}_{G,k+1}$ and $\mathbf{x}_{P,k+1}$ are obtained as

$$\begin{aligned} \mathbf{V}_{F\lambda_i} &= \frac{\partial {}^G \mathbf{p}_{f_i}}{\partial \lambda_i} \\ &= \frac{1}{\hat{\rho}_i} {}^{k+1}_G \mathbf{C}_{\hat{q}}^\top \mathbf{C}_I^\top \mathbf{C}_{\hat{q}}^\top \\ &\quad \times \begin{bmatrix} -\sin \hat{\phi}_i \sin \hat{\psi}_i & \cos \hat{\phi}_i \cos \hat{\psi}_i \\ \cos \hat{\phi}_i & 0 \\ -\sin \hat{\phi}_i \cos \hat{\psi}_i & -\cos \hat{\phi}_i \sin \hat{\psi}_i \end{bmatrix} - \frac{1}{\hat{\rho}_i} \hat{\mathbf{e}}_i \end{aligned} \quad (92)$$

$$\mathbf{V}_{FG_i} = \frac{\partial {}^G \mathbf{p}_{f_i}}{\partial \mathbf{x}_{G,k+1}} = \begin{bmatrix} -[{}^G \hat{\mathbf{p}}_{f_i}]_G^{k+1} \mathbf{C}_{\hat{q}}^\top & -{}^{k+1}_G \mathbf{C}_{\hat{q}}^\top & \mathbf{0}_3 \end{bmatrix} \quad (93)$$

$$\begin{aligned} \mathbf{V}_{FP_i} &= \frac{\partial {}^G \mathbf{p}_{f_i}}{\partial \mathbf{x}_{P,k+1}} \\ &= \begin{bmatrix} -{}^{k+1}_G \mathbf{C}_{\hat{q}}^\top \mathbf{C}_I^\top \mathbf{C}_{\hat{q}}^\top [\frac{1}{\hat{\rho}_i} \hat{\mathbf{e}}_i - {}^C \hat{\mathbf{p}}_I] & -{}^{k+1}_G \mathbf{C}_{\hat{q}}^\top \mathbf{C}_I^\top \mathbf{C}_{\hat{q}}^\top & \mathbf{0}_{3 \times 1} \end{bmatrix}. \end{aligned} \quad (94)$$

Therefore, for λ we have $\mathbf{V}_{F\lambda}$, $\mathbf{V}_{FG}$, and $\mathbf{V}_{FP}$ as

$$\begin{aligned} \mathbf{V}_{F\lambda} &= \begin{bmatrix} \mathbf{V}_{F\lambda_1} & \cdots & \mathbf{0}_3 \\ & \ddots & \vdots \\ & & \mathbf{V}_{F\lambda_s} \end{bmatrix}, \quad \mathbf{V}_{FG} = \begin{bmatrix} \mathbf{V}_{FG_1} \\ \vdots \\ \mathbf{V}_{FG_s} \end{bmatrix}, \\ \mathbf{V}_{FP} &= \begin{bmatrix} \mathbf{V}_{FP_1} \\ \vdots \\ \mathbf{V}_{FP_s} \end{bmatrix}. \end{aligned} \quad (95)$$

Then, we compute their inverse matrices to be used for the information matrix composition

$$\mathbf{U}_G^{-1} = \begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ & \mathbf{I} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ & & \mathbf{V}_{GG}^{-1} & \mathbf{0} & -\mathbf{V}_{GG}^{-1} \mathbf{V}_{GW} \\ & & & \mathbf{I} & \mathbf{0} \\ & & & & \mathbf{I} \end{bmatrix} \quad (96)$$

$$\mathbf{U}_F^{-1} = \begin{bmatrix} \mathbf{V}_{F\lambda}^{-1} & \mathbf{0} & -\mathbf{V}_{F\lambda}^{-1} \mathbf{V}_{FG} & -\mathbf{V}_{F\lambda}^{-1} \mathbf{V}_{FP} & \mathbf{0} \\ & \mathbf{I} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ & & \mathbf{I} & \mathbf{0} & \mathbf{0} \\ & & & \mathbf{I} & \mathbf{0} \\ & & & & \mathbf{I} \end{bmatrix}. \quad (97)$$

Especially, the square root information matrix $\mathbf{R}_{k+1}^+$ can be expressed in detail, as

$$\mathbf{R}_{k+1}^+ = \begin{bmatrix} \mathbf{R}_{\lambda\lambda} & \mathbf{0} & \mathbf{0} & \mathbf{R}_{\lambda P} & \mathbf{R}_{\lambda W} \\ & \mathbf{R}_{FF}^\oplus & \mathbf{R}_{FG}^\oplus & \mathbf{R}_{FP}^\oplus & \mathbf{R}_{FW}^\oplus \\ & & \mathbf{R}_{GG}^\oplus & \mathbf{R}_{GP}^\oplus & \mathbf{R}_{GW}^\oplus \\ & & & \mathbf{R}_{PP}^\oplus & \mathbf{R}_{PW}^\oplus \\ & & & & \mathbf{R}_{WW}^\oplus \end{bmatrix} \quad (98)$$

and with those Jacobian matrices we perform composition, as

$$\mathbf{R}_{k+1} = \mathbf{R}_{k+1}^+ \mathbf{U}_G^{-1} \mathbf{U}_F^{-1} = \begin{bmatrix} \mathbf{R}_{FF} & \mathbf{0} & \mathbf{R}_{FG} & \mathbf{R}_{FP} & \mathbf{R}_{\lambda W} \\ & \mathbf{R}_{FF}^\oplus & \mathbf{R}_{FG} & \mathbf{R}_{FP}^\oplus & \mathbf{R}_{FW} \\ & & \mathbf{R}_{GG} & \mathbf{R}_{GP}^\oplus & \mathbf{R}_{GW} \\ & & & \mathbf{R}_{PP}^\oplus & \mathbf{R}_{PW} \\ & & & & \mathbf{R}_{WW}^\oplus \end{bmatrix} \quad (99)$$

$$\mathbf{R}_{FF} = \mathbf{R}_{\lambda\lambda} \mathbf{V}_{F\lambda}^{-1} \quad (100)$$

$$\mathbf{R}_{FG} = -\mathbf{R}_{\lambda\lambda} \mathbf{V}_{F\lambda}^{-1} \mathbf{V}_{FG} \quad (101)$$

$$\mathbf{R}_{FP} = \mathbf{R}_{\lambda P} - \mathbf{R}_{\lambda\lambda} \mathbf{V}_{F\lambda}^{-1} \mathbf{V}_{FP} \quad (102)$$

$$\mathbf{R}_{FG} = \mathbf{R}_{FG}^\oplus \mathbf{V}_{GG}^{-1} \quad (103)$$

$$\mathbf{R}_{FW} = \mathbf{R}_{FW}^\oplus - \mathbf{R}_{FG}^\oplus \mathbf{V}_{GG}^{-1} \mathbf{V}_{GW} \quad (104)$$

$$\mathbf{R}_{GG} = \mathbf{R}_{GG}^\oplus \mathbf{V}_{GG}^{-1} \quad (105)$$

$$\mathbf{R}_{GW} = \mathbf{R}_{GW}^\oplus - \mathbf{R}_{GG}^\oplus \mathbf{V}_{GG}^{-1} \mathbf{V}_{GW}. \quad (106)$$

APPENDIX E

EXPRESSIONS OF JACOBIANS IN (33)

The components of Jacobians $\mathbf{H}_{f_i}$ and $\mathbf{H}_{f_j}$ are as follows:

$$\mathbf{H}_{f_i} = \frac{\partial \mathbf{e}_{f_{ij}}}{\partial \mathbf{x}_{I_i}} = \mathbf{H}_{\text{proj}} \begin{bmatrix} \mathbf{H}_{\theta_i} & \mathbf{H}_{\mathbf{p}_i} & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 \end{bmatrix} \quad (107)$$

$$\mathbf{H}_{\text{proj}} = \frac{\partial \mathbf{e}_{f_{ij}}}{\partial \mathbf{C}_i \mathbf{p}_{f_j}} = \hat{\mathbf{z}}_{j,i}^{-1} \begin{bmatrix} 1 & 0 & -\hat{x}_{j,i} \hat{z}_{j,i}^{-1} \\ 0 & 1 & -\hat{y}_{j,i} \hat{z}_{j,i}^{-1} \end{bmatrix} \quad (108)$$

$$\mathbf{H}_{\theta_i} = \frac{\partial \mathbf{C}_i \mathbf{p}_{f_j}}{\partial \delta_G^i \theta} = {}^C_I \mathbf{C}_{\hat{q}}^i [{}^G \mathbf{C}_{\hat{q}}^i ({}^G \hat{\mathbf{p}}_{f_j} - {}^G \hat{\mathbf{p}}_{I_i})] \quad (109)$$

$$\mathbf{H}_{\mathbf{p}_i} = \frac{\partial \mathbf{C}_i \mathbf{p}_{f_j}}{\partial \mathbf{p}_{I_i}} = -{}^C_I \mathbf{C}_{\hat{q}}^i {}^G \mathbf{C}_{\hat{q}}^i \quad (110)$$

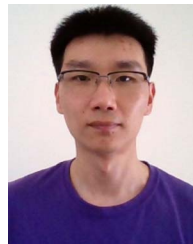
and

$$\mathbf{H}_{f_j} = \frac{\partial \mathbf{e}_{f_{ij}}}{\partial \mathbf{x}_{f_j}} = \mathbf{H}_{\text{proj}} \mathbf{H}_{\mathbf{p}_j}, \quad \mathbf{H}_{\mathbf{p}_j} = \frac{\partial \mathbf{C}_i \mathbf{p}_{f_j}}{\partial \mathbf{p}_{f_j}} = {}^C_I \mathbf{C}_{\hat{q}}^i {}^G \mathbf{C}_{\hat{q}}^i. \quad (111)$$

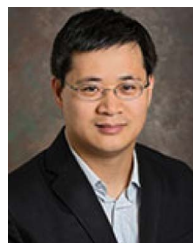
REFERENCES

- [1] R. C. Smith and P. Cheeseman, "On the representation and estimation of spatial uncertainty," *Int. J. Robot. Res.*, vol. 5, no. 4, pp. 56–68, 1986.
- [2] P. Cheeseman, R. Smith, and M. Self, "A stochastic map for uncertain spatial relationships," in *Proc. Int. Symp. Robot. Res.*, MIT Press, Cambridge, 1987, pp. 467–474.
- [3] J. J. Leonard and H. F. Durrant-Whyte, "Simultaneous map building and localization for an autonomous mobile robot," in *Proc. IEEE/RSJ Int. Workshop Intell. Robots Syst.*, 1991, vol. 3, pp. 1442–1447.
- [4] K. J. Wu, C. X. Guo, G. Georgiou, and S. I. Roumeliotis, "VINS on wheels," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 5155–5162.
- [5] J. D. Tardós, J. Neira, P. M. Newman, and J. J. Leonard, "Robust mapping and localization in indoor environments using sonar data," *Int. J. Robot. Res.*, vol. 21, no. 4, pp. 311–330, 2002.
- [6] J. Zhang and S. Singh, "LOAM: Lidar odometry and mapping in real-time," *Robot. Sci. Syst.*, vol. 2, no. 9, pp. 1–9, Jul. 2014.
- [7] J. Zhang and S. Singh, "Low-drift and real-time Lidar odometry and mapping," *Auton. Robots*, vol. 41, pp. 401–416, 2017.
- [8] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse, "MonoSLAM: Real-time single camera SLAM," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 29, no. 6, pp. 1052–1067, Jun. 2007.
- [9] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.
- [10] S. I. Roumeliotis, A. E. Johnson, and J. F. Montgomery, "Augmenting inertial navigation with image-based motion estimation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2002, vol. 4, pp. 4326–4333.
- [11] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-manifold preintegration for real-time visual-inertial odometry," *IEEE Trans. Robot.*, vol. 33, no. 1, pp. 1–21, Feb. 2017.
- [12] P. Geneva, K. Eickenhoff, Y. Yang, and G. Huang, "LiPS: LIDAR-inertial 3D plane SLAM," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 123–130.
- [13] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct LiDAR-inertial odometry," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2053–2073, Aug. 2022.
- [14] A. I. Mourikis and S. I. Roumeliotis, "A multi-state constraint Kalman filter for vision-aided inertial navigation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2007, pp. 3565–3572.
- [15] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, "Keyframe-based visual-inertial odometry using nonlinear optimization," *Int. J. Robot. Res.*, vol. 34, no. 3, pp. 314–334, 2015.
- [16] Z. Huai and G. Huang, "Robocentric visual-inertial odometry," *Int. J. Robot. Res.*, vol. 41, no. 7, pp. 667–689, 2022.
- [17] P. S. Maybeck, *Stochastic Models, Estimation, and Control*. Cambridge, MA, USA: Academic Press, 1982.
- [18] B. Triggs, P. F. McLauchlan, R. I. Hartley, and A. W. Fitzgibbon, "Bundle adjustment—A modern synthesis," in *Proc. Int. Workshop Vis. Algorithms*. Springer, 1999, pp. 298–372.
- [19] F. Dellaert and M. Kaess, "Square root SAM: Simultaneous localization and mapping via square root information smoothing," *Int. J. Robot. Res.*, vol. 25, no. 12, pp. 1181–1203, 2006.
- [20] M. Kaess, A. Ranganathan, and F. Dellaert, "iSAM: Incremental smoothing and mapping," *IEEE Trans. Robot.*, vol. 24, no. 6, pp. 1365–1378, Dec. 2008.
- [21] T. Ke, K. J. Wu, and S. I. Roumeliotis, "Rise-SLAM: A resource-aware inverse schmidt estimator for SLAM," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 354–361.
- [22] K. Wu, A. Ahmed, G. A. Georgiou, and S. I. Roumeliotis, "A square root inverse filter for efficient vision-aided inertial navigation on mobile devices," *Robot. Sci. Syst.*, vol. 2, p. 2, 2015.
- [23] G. Klein and D. Murray, "Parallel tracking and mapping for small ar workspaces," in *Proc. IEEE ACM Int. Symp. Mixed Augmented Reality*, 2007, pp. 225–234.
- [24] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-scale direct monocular SLAM," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2014, pp. 834–849.
- [25] H. Liu, M. Chen, G. Zhang, H. Bao, and Y. Bao, "ICE-BA: Incremental, consistent and efficient bundle adjustment for visual-inertial SLAM," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 1974–1982.
- [26] Z. Huai and G. Huang, "Markov parallel tracking and mapping for probabilistic SLAM," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 11661–11667.
- [27] C. Cadena et al., "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1309–1332, Dec. 2016.
- [28] G. Huang, "Visual-inertial navigation: A concise review," in *Proc. Int. Conf. Robot. Automat.*, 2019, pp. 9572–9582.
- [29] L. Kneip, M. Chli, and R. Siegwart, "Robust real-time visual odometry with a single camera and an IMU," in *Proc. Brit. Mach. Vis. Conf.*, British Machine Vision Association, 2011.
- [30] S. Weiss and R. Siegwart, "Real-time metric state estimation for modular vision-inertial systems," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2011, pp. 4531–4537.
- [31] H. Durrant-Whyte and T. Bailey, "Simultaneous localization and mapping: Part I," *IEEE Robot. Automat. Mag.*, vol. 13, no. 2, pp. 99–110, Jun. 2006.
- [32] T. Bailey and H. Durrant-Whyte, "Simultaneous localization and mapping (SLAM): Part II," *IEEE Robot. Automat. Mag.*, vol. 13, no. 3, pp. 108–117, Sep. 2006.
- [33] M. Li and A. I. Mourikis, "High-precision, consistent EKF-based visual-inertial odometry," *Int. J. Robot. Res.*, vol. 32, no. 6, pp. 690–711, 2013.

- [34] M. Bloesch, M. Burri, S. Omari, M. Hutter, and R. Siegwart, "Iterated extended Kalman filter based visual-inertial odometry using direct photometric feedback," *Int. J. Robot. Res.*, vol. 36, no. 10, pp. 1053–1072, 2017.
- [35] P. Geneva, K. Eickenhoff, W. Lee, Y. Yang, and G. Huang, "OpenVINS: A research platform for visual-inertial estimation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 4666–4672.
- [36] T.-C. Dong-Si and A. I. Mourikis, "Motion tracking with fixed-lag smoothing: Algorithm and consistency analysis," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2011, pp. 5655–5662.
- [37] M. Li and A. I. Mourikis, "Optimization-based estimator design for vision-aided inertial navigation," in *Robotics: Sci. Syst.*, Berlin Germany, 2013, pp. 241–248.
- [38] J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 3, pp. 611–625, Mar. 2018.
- [39] V. Usenko, J. Engel, J. Stückler, and D. Cremers, "Direct visual-inertial odometry with stereo cameras," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2016, pp. 1885–1892.
- [40] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.
- [41] T. Qin, P. Li, and S. Shen, "VINS-Mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.
- [42] S. Leutenegger, "OKVIS2: Realtime scalable visual-inertial slam with loop closure," 2022, *arXiv:2202.09199*.
- [43] M. Kaess, H. Johannsson, R. Roberts, V. Ila, J. J. Leonard, and F. Dellaert, "iSAM2: Incremental smoothing and mapping using the Bayes tree," *Int. J. Robot. Res.*, vol. 31, no. 2, pp. 216–235, 2012.
- [44] M. Kaess, V. Ila, R. Roberts, and F. Dellaert, "The Bayes tree: An algorithmic foundation for probabilistic robot mapping," in *Algorithmic Foundations of Robotics IX*. Springer, Berlin Heidelberg, Germany, 2010, pp. 157–173.
- [45] P. Geneva, J. Maley, and G. Huang, "An efficient schmidt-EKF for 3D visual-inertial SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 12105–12115.
- [46] S. J. Julier and J. K. Uhlmann, "A non-divergent estimation algorithm in the presence of unknown correlations," in *Proc. IEEE Amer. Control Conf.*, 1997, vol. 4, pp. 2369–2373.
- [47] K. Wu, T. Zhang, D. Su, S. Huang, and G. Dissanayake, "An invariant-EKF vins algorithm for improving consistency," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2017, pp. 1578–1585.
- [48] M. Kaess, S. Williams, V. Indelman, R. Roberts, J. J. Leonard, and F. Dellaert, "Concurrent filtering and smoothing," in *Proc. 15th Int. Conf. Inf. Fusion*, 2012, pp. 1300–1307.
- [49] S. Williams, V. Indelman, M. Kaess, R. Roberts, J. J. Leonard, and F. Dellaert, "Concurrent filtering and smoothing: A parallel architecture for real-time navigation and full smoothing," *Int. J. Robot. Res.*, vol. 33, no. 12, pp. 1544–1568, 2014.
- [50] Y. Zhang, M. Hsiao, Y. Zhao, J. Dong, and J. J. Engel, "Distributed client-server optimization for SLAM with limited on-device resources," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 5336–5342.
- [51] G. H. Golub and C. F. Van Loan, *Matrix Computations*, vol. 3, Baltimore, MD, USA: JHU Press, 2012.
- [52] J. Vial, H. Durrant-Whyte, and T. Bailey, "Conservative sparsification for efficient and consistent approximate estimation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2011, pp. 886–893.
- [53] G. Huang, M. Kaess, and J. J. Leonard, "Consistent sparsification for graph optimization," in *Proc. Eur. Conf. Mobile Robots*, 2013, pp. 150–157.
- [54] G. Huang, A. I. Mourikis, and S. I. Roumeliotis, "An observability constrained sliding window filter for SLAM," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, San Francisco, CA, 2011, pp. 65–72.
- [55] K. J. Wu and S. I. Roumeliotis, "Inverse schmidt estimators," University of Minnesota, MARS, Tech. Rep., 2016, <http://mars.cs.umn.edu/tr/inverseschmidt.pdf>.
- [56] Z. Huai and G. Huang, "Square-root robocentric visual-inertial odometry with online spatiotemporal calibration," *IEEE Robot. Automat. Lett.*, vol. 7, no. 4, pp. 9961–9968, Oct. 2022.
- [57] J. Civera, A. J. Davison, and J. M. Montiel, "Inverse depth parametrization for monocular SLAM," *IEEE Trans. Robot.*, vol. 24, no. 5, pp. 932–945, Oct. 2008.
- [58] G. P. Huang, A. I. Mourikis, and S. I. Roumeliotis, "Observability-based rules for designing consistent EKF SLAM estimators," *Int. J. Robot. Res.*, vol. 29, no. 5, pp. 502–528, 2010.
- [59] N. Trawny and S. I. Roumeliotis, "Indirect Kalman filter for 3D attitude estimation," University of Minnesota, MARS, Tech. Rep., 2005. [Online]. Available: <https://mars.cs.umn.edu/tr/reports/Trawny05b.pdf>
- [60] W. Breckenridge, "Quaternions proposed standard conventions," *Jet Propulsion Lab.*, Pasadena, CA, IOM 343-79-1199, 1979.
- [61] T. Schneider et al., "Maplab: An open framework for research in visual-inertial mapping and localization," *IEEE Robot. Automat. Lett.*, vol. 3, no. 3, pp. 1418–1425, Jul. 2018.
- [62] K. Eickenhoff, P. Geneva, and G. Huang, "Closed-form preintegration methods for graph-based visual-inertial navigation," *Int. J. Robot. Res.*, vol. 38, no. 5, pp. 563–586, 2019.
- [63] S. Agarwal, K. Mierle, and T. C. S. Team, "Ceres solver," 3, 2022. [Online]. Available: <https://github.com/ceres-solver/ceres-solver>
- [64] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation With Applications to Tracking and Navigation*. Hoboken, NJ, USA: Wiley, 2004.
- [65] G. Guennebaud et al., "Eigen," vol. 3, no. 1, 2010. [Online]. Available: <http://eigen.tuxfamily.org>
- [66] D. Gálvez-López and J. D. Tardos, "Bags of binary words for fast place recognition in image sequences," *IEEE Trans. Robot.*, vol. 28, no. 5, pp. 1188–1197, Oct. 2012.
- [67] M. Calonder, V. Lepetit, C. Strecha, and P. Fua, "Brief: Binary robust independent elementary features," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2010, pp. 778–792.
- [68] M. Burri et al., "The EuRoC micro aerial vehicle datasets," *Int. J. Robot. Res.*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [69] J. Shi et al., "Good features to track," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 1994, pp. 593–600.
- [70] Z. Zhang and D. Scaramuzza, "A tutorial on quantitative trajectory evaluation for visual (-inertial) odometry," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 7244–7251.
- [71] K. Wu, "On the efficiency and consistency of visual-inertial localization and mapping," Ph.D. dissertation, Univ. Minnesota, 2024.



Zheng Huai (Member, IEEE) received the B.Eng. degree in automation and the M.S. degree in control theory and control engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2009 and 2013, respectively, and the Ph.D. degree in mechanical engineering from the University of Delaware, Newark, DE, USA, in 2023. Since 2022, he has been working on SLAM, computer vision, and hardware acceleration technologies with Google LLC, Mountain View, CA, USA, for AR/VR/MR platforms and services (Google XR).



Guoquan Huang (Senior Member, IEEE) received the B.S. degree in automation (electrical engineering) from the University of Science and Technology Beijing, China, in 2002, and the M.S. and Ph.D. degrees in computer science from the University of Minnesota–Twin Cities, Minneapolis, MN, USA, in 2009 and 2012, respectively.

He is currently an Associate Professor of mechanical engineering and computer and information sciences with the University of Delaware (UD), where he is leading Robot Perception and Navigation Group (RPNP). From 2012 to 2014, he was a Postdoctoral Associate with MIT CSAIL (Marine Robotics), Cambridge, MA, USA. His research interests include state estimation and spatial computing for autonomous robots and mobile devices, including probabilistic sensing, estimation, localization, mapping, perception, locomotion, and navigation.

Dr. Huang was the recipient of the 2015 UD Research Award (UDRF), 2015/2023 NASA DE Space Research Seed Award, 2016 NSF Research Initiation (CRII) Award, 2018/2019/2022 Google Daydream/ARVR/AI Faculty Research Award, ICRA 2022 Best Paper Award (Navigation), 2023 GNC Best Overseas Paper, 2023 Meta Reality Labs Faculty Research Award, and the Finalists of ICRA 2024 Best Paper Award (Robot Vision), RSS 2023 Best Student Paper Award, ICRA 2021 Best Paper Award (Robot Vision), and RSS 2009 Best Paper Award. He has served as an Associate Editor for IEEE TRANSACTIONS ON ROBOTICS (T-RO) and IET Cyber-Systems and Robotics (CSR), as well as previously for the IEEE Robotics and Automation Letters (RA-L).